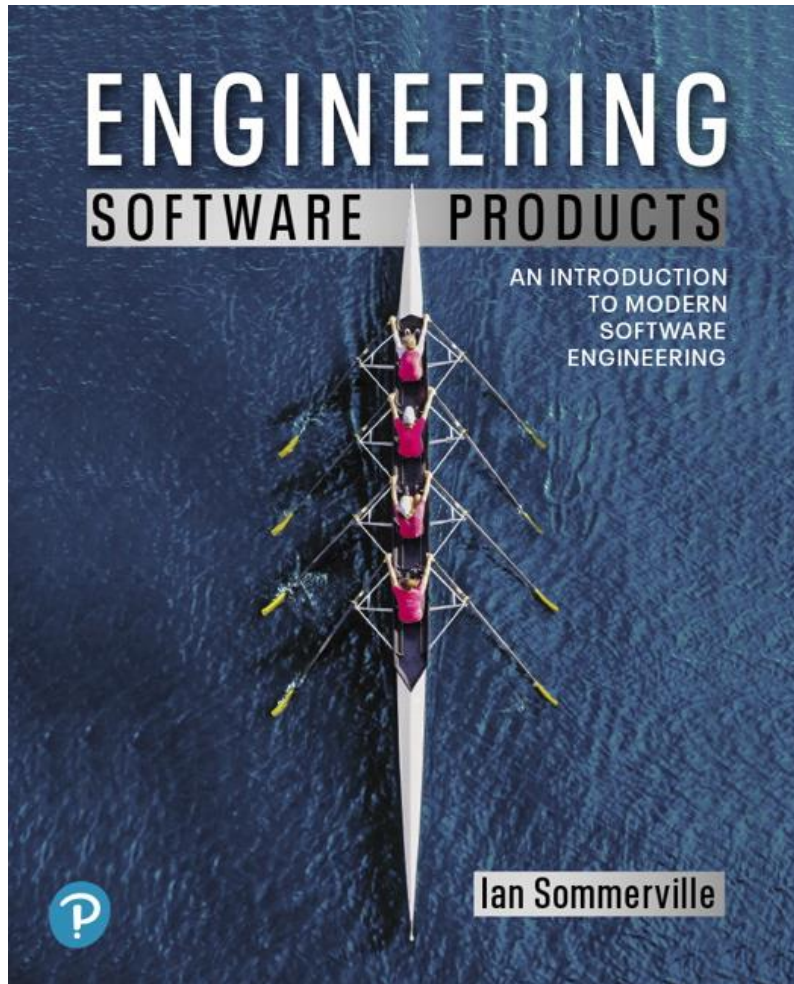


Engineering Software Products

First Edition



Chapter 4

Software Architecture Part 1

Software architecture (1 of 2)

- To create a reliable, secure and efficient product, you need to pay attention to architectural design which includes:
 - its overall organization,
 - how the software is decomposed into components,
 - the server organization
 - the technologies that you use to build the software
 - The architecture of a software product affects its performance, usability, security, reliability and maintainability.

Software architecture (2 of 2)

- There are many different interpretations of the term 'software architecture'.
 - Some focus on 'architecture' as a noun - the structure of a system and others consider 'architecture' to be a verb - the process of defining these structures.

Table 4.1 The IEEE definition of software architecture

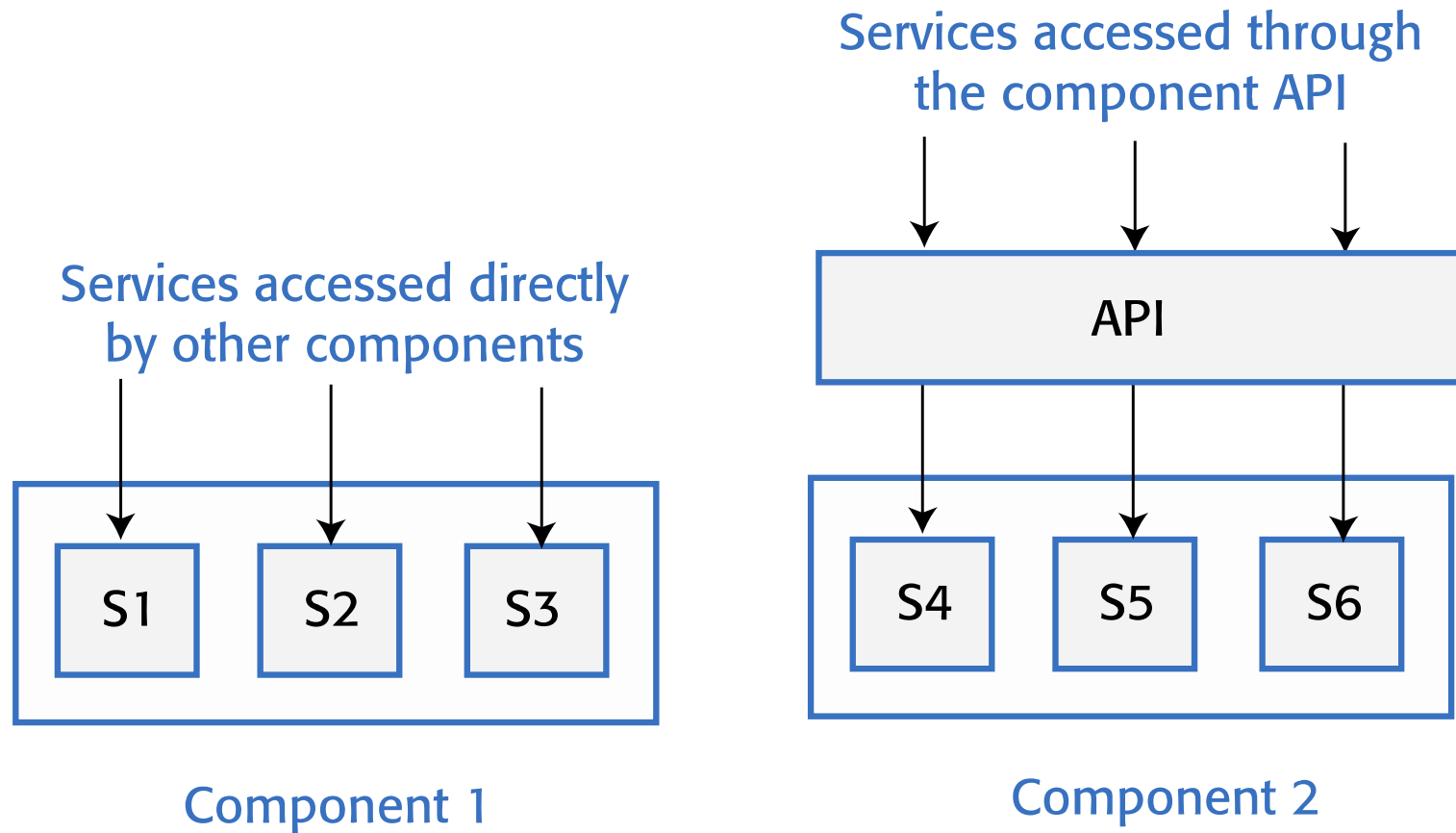
Software architecture

Architecture is the fundamental organization of a software system embodied in its components, their relationships to each other and to the environment, and the principles guiding its design and evolution.

Software architecture and components

- A component is an element that implements a coherent set of functionality or features.
- Software component can be considered as a collection of one or more services that may be used by other components.
- When designing software architecture, you don't have to decide how an architectural element or component is to be implemented.
- Rather, you design the component interface and leave the implementation of that interface to a later stage of the development process.

Figure 4.1



Access to services provided by software components

Why is architecture important? (1 of 2)

- Architecture is important because the architecture of a system has a fundamental influence on the non-functional system properties, shown in Table 4.2.
- Architectural design involves understanding the issues that affect the architecture of your product and creating an architectural description that shows the critical components and their relationships.

Why is architecture important? (2 of 2)

- Minimizing complexity should be an important goal for architectural designers.
 - The more complex a system, the more difficult and expensive it is to understand and change.
 - Programmers are more likely to make mistakes and introduce bugs and security vulnerabilities when they are modifying or extending a complex system..

Table 4.2 Non-functional system quality attributes

Attribute	Key issue
Responsiveness	Does the system return results to users in a reasonable time?
Reliability	Do the system features behave as expected by both developers and users?
Availability	Can the system deliver its services when requested by users?
Security	Does the system protect itself and users' data from unauthorized attacks and intrusions?
Usability	Can system users access the features that they need and use them quickly and without errors?
Maintainability	Can the system be readily updated and new features added without undue costs?
Resilience	Can the system continue to deliver user services in the event of partial failure or external attack?

Centralized Security

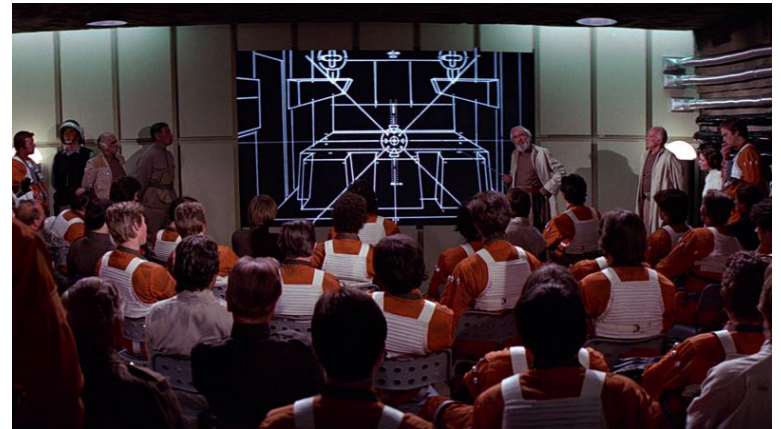
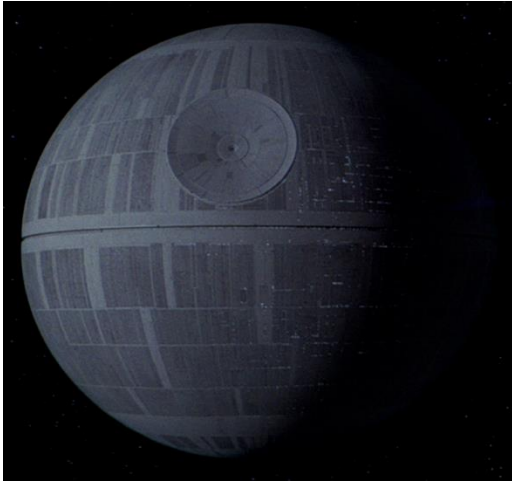




Table 4.3 The influence of architecture on system security

A centralized security architecture

In the *Star Wars* prequel *Rogue One* (https://en.wikipedia.org/wiki/Rogue_One), the evil Empire has stored the plans for all of their equipment in a single, highly secure, wellguarded, remote location. This is called a centralized security architecture. It is based on the principle that if you maintain all of your information in one place, then you can apply lots of resources to protect that information and ensure that intruders can't get it.

Unfortunately (for the Empire), the rebels managed to breach their security. They stole the plans for the Death Star, an event that underpins the whole *Star Wars* saga. In trying to stop them, the Empire destroyed their entire archive of system documentation with who knows what resultant costs. Had the Empire chosen a distributed security architecture, with different parts of the Death Star plans stored in different locations, then stealing the plans would have been more difficult. The rebels would have had to breach security in all locations to steal the complete Death Star blueprints.

Centralized security architectures benefits

- The benefits of a centralized security architecture are that it is easier to design and build protection and that the protected information can be accessed more efficiently.

Centralized security architectures drawbacks

If you distribute information, it takes longer to access all of the information and costs more to protect it.

If security is breached in one location, you only lose the information that you have stored there.

However, if your security is breached, you lose everything.



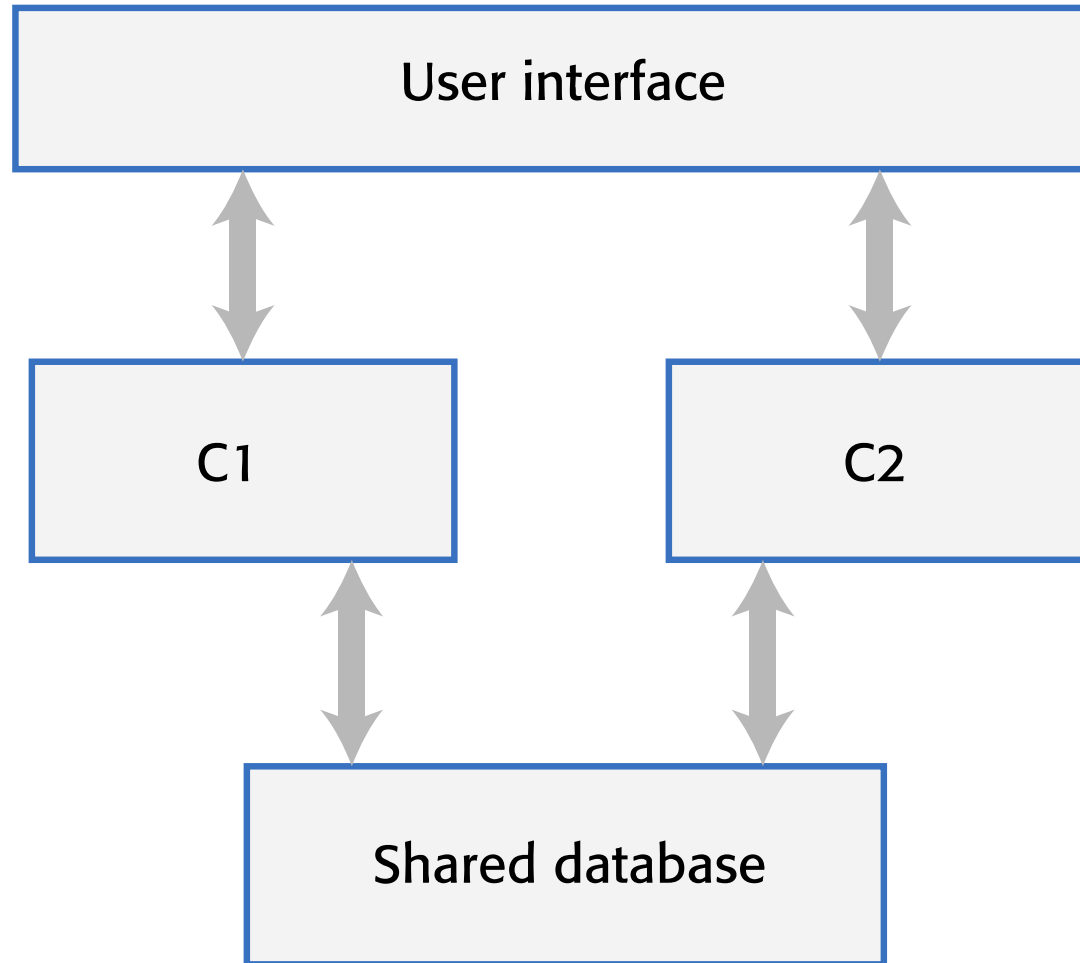
Maintainability and performance (1 of 2)

- Figure 4.2 shows a system with two components (C1 and C2) that share a common database.
 - Assume C1 runs slowly because it has to reorganize the information in the database before using it.
 - The only way to make C1 faster might be to change the database. This means that C2 also has to be changed, which may, potentially, affect its response time.
- In Figure 4.3, a different architecture is used where each component has its own copy of the parts of the database that it needs.
 - If one component needs to change the database organization, this does not affect the other component.

Maintainability and performance (2 of 2)

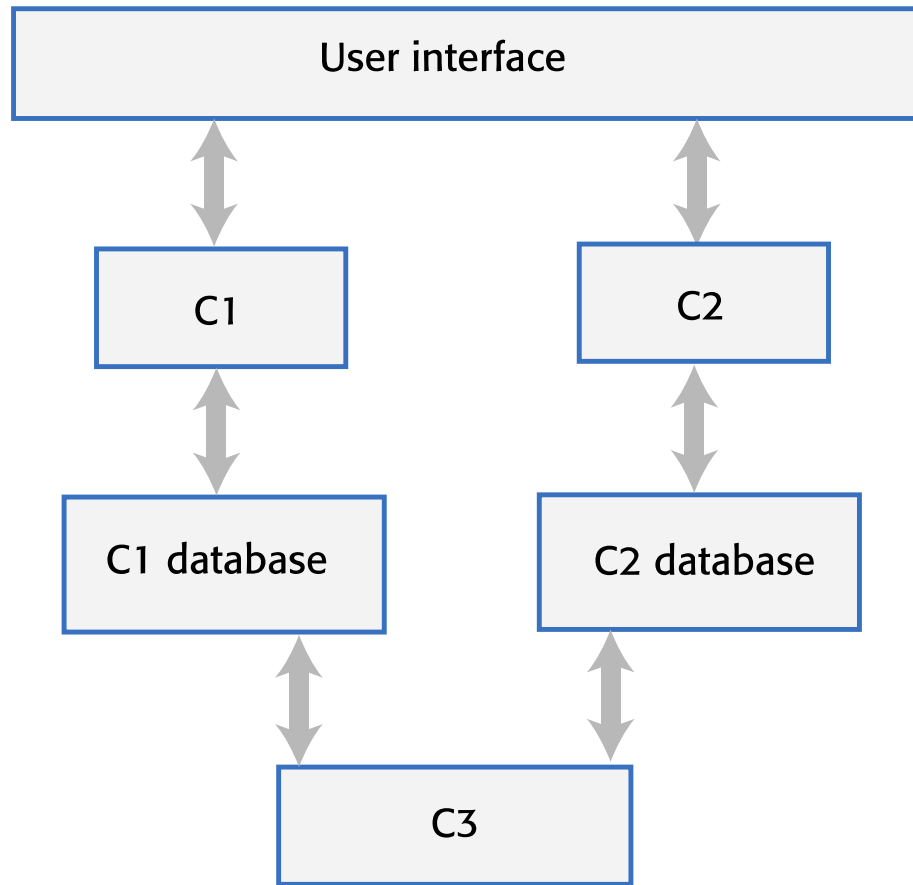
- However, a multi-database architecture may run more slowly and may cost more to implement and change.
 - A multi-database architecture needs a mechanism (component C3) to ensure that the data shared by C1 and C2 is kept consistent when it is changed.

Figure 4.2



Shared database architecture

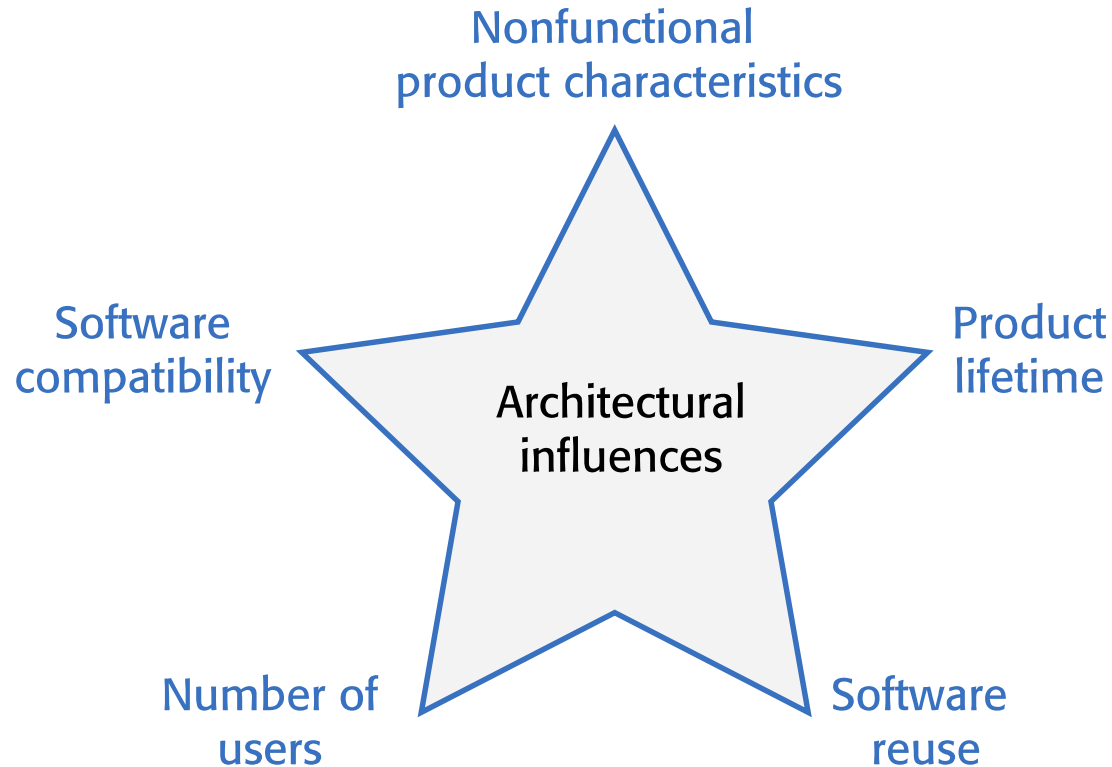
Figure 4.3



Database reconciliation

Multiple database architecture

Figure 4.4



Issues that influence architectural decisions

Table 4.4 The importance of architectural design issues (1 of 2)

Issue	Architectural importance
Non-functional product characteristics	Non-functional product characteristics such as security and performance affect all users. If you get these wrong, your product is unlikely to be a commercial success. Unfortunately, some characteristics are opposing, so you can optimize only the most important.
Product lifetime	If you anticipate a long product lifetime, you need to create regular product revisions. You therefore need an architecture that can evolve, so that it can be adapted to accommodate new features and technology.
Software reuse	You can save a lot of time and effort if you can reuse large components from other products or open-source software. However, this constrains your architectural choices because you must fit your design around the software that is being reused.

Table 4.4 The importance of architectural design issues (2 of 2)

Issue	Architectural importance
Number of users	If you are developing consumer software delivered over the Internet, the number of users can change very quickly. This can lead to serious performance degradation unless you design your architecture so that your system can be quickly scaled up and down.
Software compatibility	For some products, it is important to maintain compatibility with other software so that users can adopt your product and use data prepared using a different system. This may limit architectural choices, such as the database software that you can use.

Trade off: Maintainability vs performance (1 of 2)

- System maintainability is an attribute that reflects how difficult and expensive it is to make changes to a system after it has been released to customers.
 - You improve maintainability by building a system from small self-contained parts, each of which can be replaced or enhanced if changes are required.

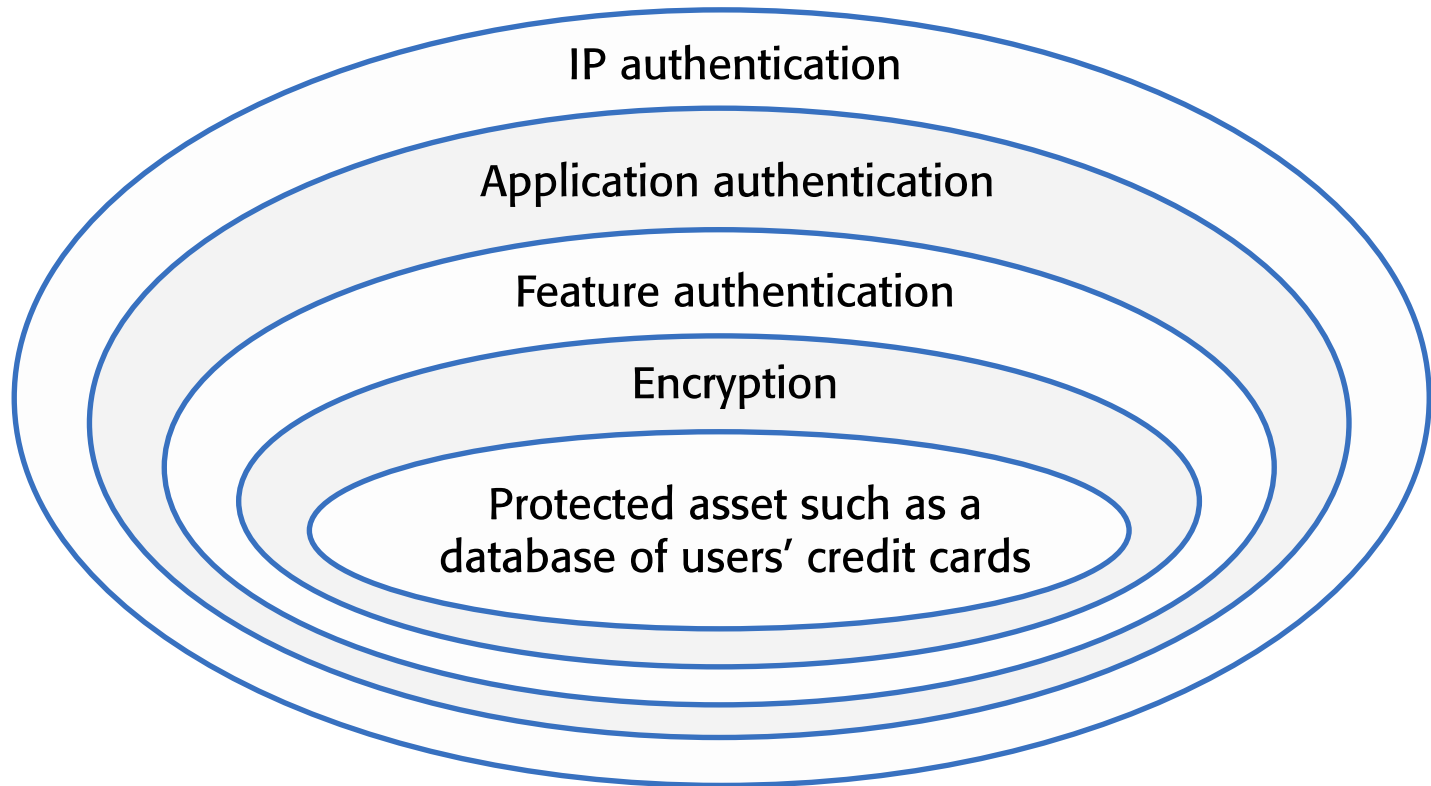
Trade off: Maintainability vs performance (2 of 2)

- In architectural terms, this means that the system should be decomposed into fine-grain components, each of which does one thing and one thing only.
 - However, it takes time for components to communicate with each other. Consequently, if many components are involved in implementing a product feature, the software will be slower.

Trade off: Security vs usability

- You can achieve security by designing the system protection as a series of layers (Figure 4.5).
 - An attacker has to penetrate all of those layers before the system is compromised.
- Layers might include system authentication layers, a separate critical feature authentication layer, an encryption layer and so on.
- Architecturally, you can implement each of these layers as separate components so that if one of these components is compromised by an attacker, then the other layers remain intact.

Figure 4.5



Authentication layers

Usability issues (1 of 2)

- A layered approach to security affects the usability of the software.
 - Users have to remember information, like passwords, that is needed to penetrate a security layer. Their interaction with the system is inevitably slowed down by its security features.
 - Many users find this irritating and often look for work-arounds so that they do not have to re-authenticate to access system features or data.

Usability issues (2 of 2)

- To avoid this, you need an architecture:
 - that doesn't have too many security layers,
 - that doesn't enforce unnecessary security,
 - that provides helper components that reduce the load on users.

Trade off: Availability vs time-to-market

(1 of 2)

- Availability is particularly important in enterprise products, such as products for the finance industry, where 24/7 operation is expected.
- The availability of a system is a measure of the amount of ‘uptime’ of that system.
 - Availability is normally expressed as a percentage of the time that a system is available to deliver user services.

Trade off: Availability vs time-to-market

(2 of 2)

- Architecturally, you achieve availability by having redundant components in a system.
 - To make use of redundancy, you include sensor components that detect failure, and switching components that switch operation to a redundant component when a failure is detected.
- Implementing extra components takes time and increases the cost of system development. It adds complexity to the system and therefore increases the chances of introducing bugs and vulnerabilities.

Architectural design questions

- How should the system be organized as a set of architectural components, where each of these components provides a subset of the overall system functionality?
 - The organization should deliver the system security, reliability and performance that you need.
- How should these architectural components be distributed and communicate with each other?
- What technologies should you use in building the system and what components should be reused?

Component organization

- Abstraction in software design means that you focus on the essential elements of a system or software component without concern for its details.
- At the architectural level, your concern should be on large-scale architectural components.
- Decomposition involves analysing these large-scale components and representing them as a set of finer-grain components.
- Layered models are often used to illustrate how a system is composed of components.

Figure 4.6

Web browser

User interaction	Local input validation	Local printing
------------------	------------------------	----------------

User interface management

Authentication and authorization	Form and query manager	Web page generation
----------------------------------	------------------------	---------------------

Information retrieval

Search	Document retrieval	Rights management	Payments	Accounting
--------	--------------------	-------------------	----------	------------

Document index

Index management	Index querying	Index creation
------------------	----------------	----------------

Basic services

Database query	Query validation	Logging	User account management
----------------	------------------	---------	-------------------------

Databases

DB1	DB2	DB3	DB4	DB5
-----	-----	-----	-----	-----

An architectural model of a document retrieval system

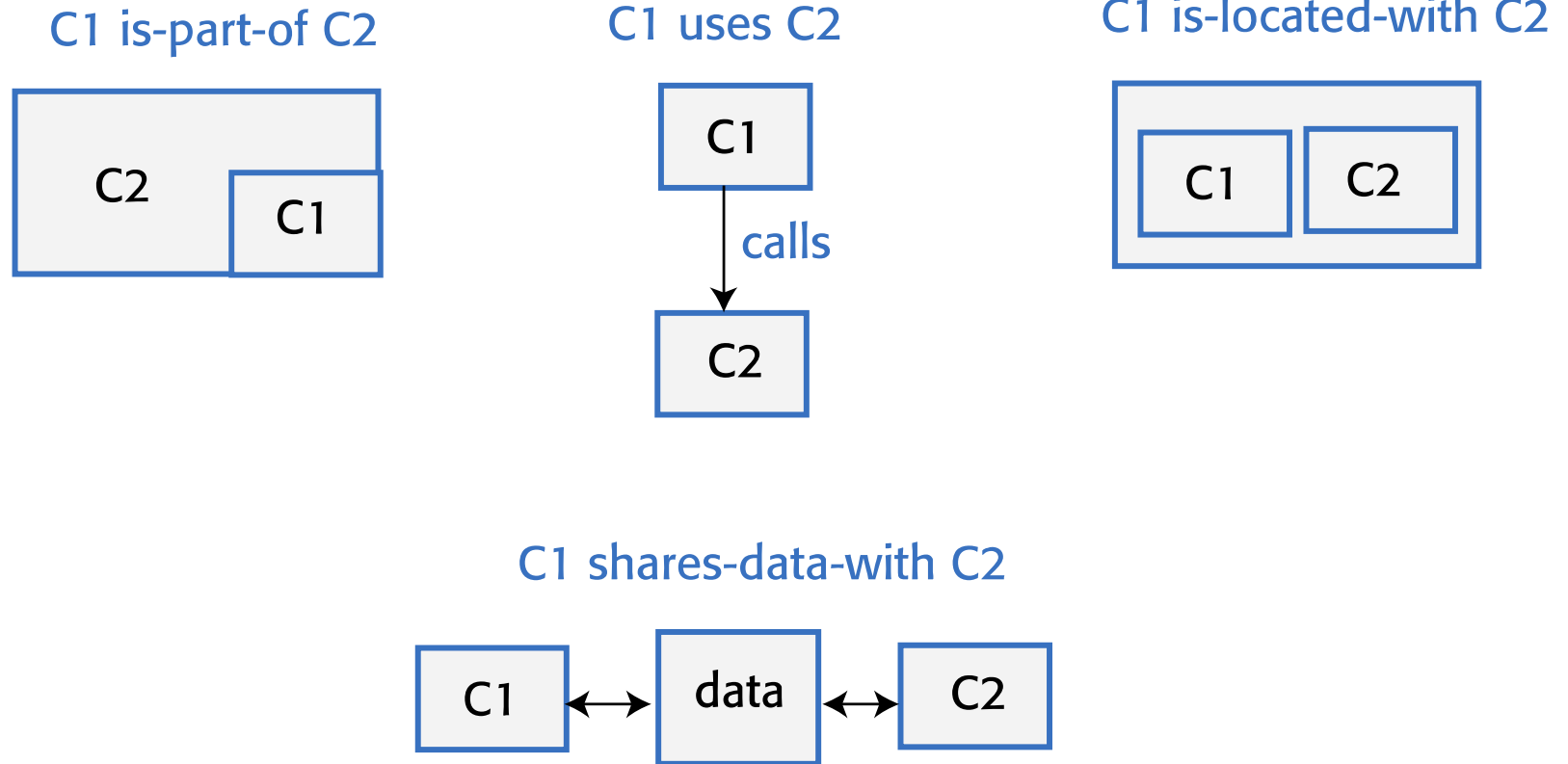
Architectural complexity (1 of 2)

- Complexity in a system architecture arises because of the number and the nature of the relationships between components in that system.
- When decomposing a system into components, you should try to avoid unnecessary software complexity.
 - *Localize relationships*
If there are relationships between components A and B, these are easier to understand if A and B are defined in the same module.

Architectural complexity (2 of 2)

- When decomposing a system into components, you should try to avoid unnecessary software complexity.
 - *Reduce shared dependencies*
Where components A and B depend on some other component or data, complexity increases because changes to the shared component mean you have to understand how these changes affect both A and B.
- It is always preferable to use local data wherever possible and to avoid sharing data if you can.

Figure 4.7



Examples of component relationships

Programming principles (1 of 2)

- Separation of concerns
 - This means that each abstraction in the program (class, method, etc.) should address a separate concern and that all aspects of that concern should be covered there. For example, if authentication is a concern in your program, then everything to do with authentication should be in one place, rather than distributed throughout your code.

Programming principles (2 of 2)

- Separate the ‘what’ from the ‘how’
 - If a program component provides a particular service, you should make available only the information that is required to use that service (the ‘what’). The implementation of the service (‘the how’) should be of no interest to service users.

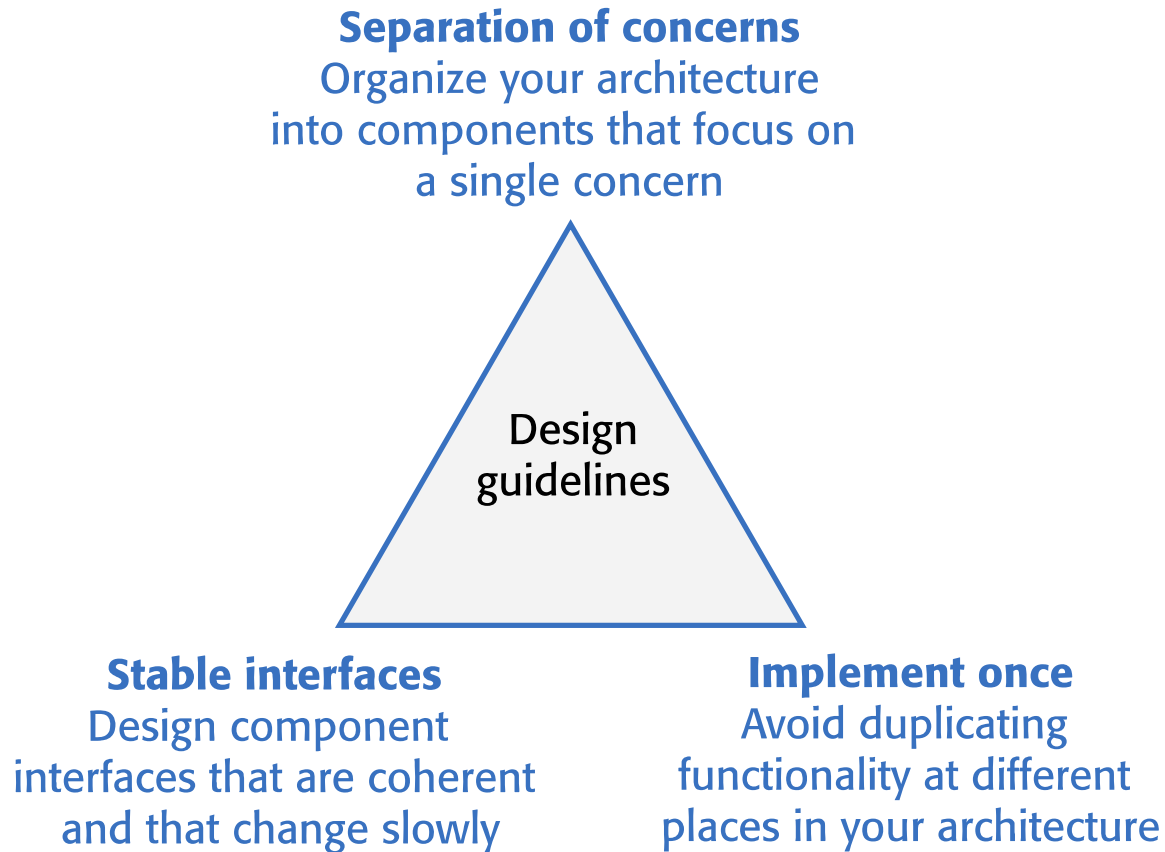
Cohesion vs coupling

- Cohesion is the measure of how related a group of things is – how strong the relationship is between the methods and data of a class
 - Coupling is basically a measure of dependence between two or more classes, modules, or components
- **We want code to that is highly cohesive and has low coupling**

High cohesion and low coupling is easier on our brains

- Better code clarity, it's much easier to understand what's going on when everything is neatly organized
- Better code reusability – when you reuse code, it's costs less to maintain it, it's also a lot easier to extend or to fix bugs because the code is only found in one place
- Better testability – when everything is neatly isolated, testing is much easier
 - You don't have to set up a whole testing environment, you can simply mock neighboring modules with dummy data
 - Can test your module as a black box by verifying just the output, or as a white box by verifying which methods are being run
- Faster Development – isolating modules helps in creating/updating new features
- Organizational ease – when everything is neatly separated engineers can have a better development experience, by agreeing on which modules to work on and not to interfere with each other

Figure 4.8



Architectural design guidelines

Design guidelines and layered architectures (1 of 2)

- Each layer is an area of concern and is considered separately from other layers.
 - The top layer is concerned with user interaction, the next layer down with user interface management, the third layer with information retrieval and so on.
- Within each layer, the components are independent and do not overlap in functionality.
 - The lower layers include components that provide general functionality so there is no need to replicate this in the components in a higher level.

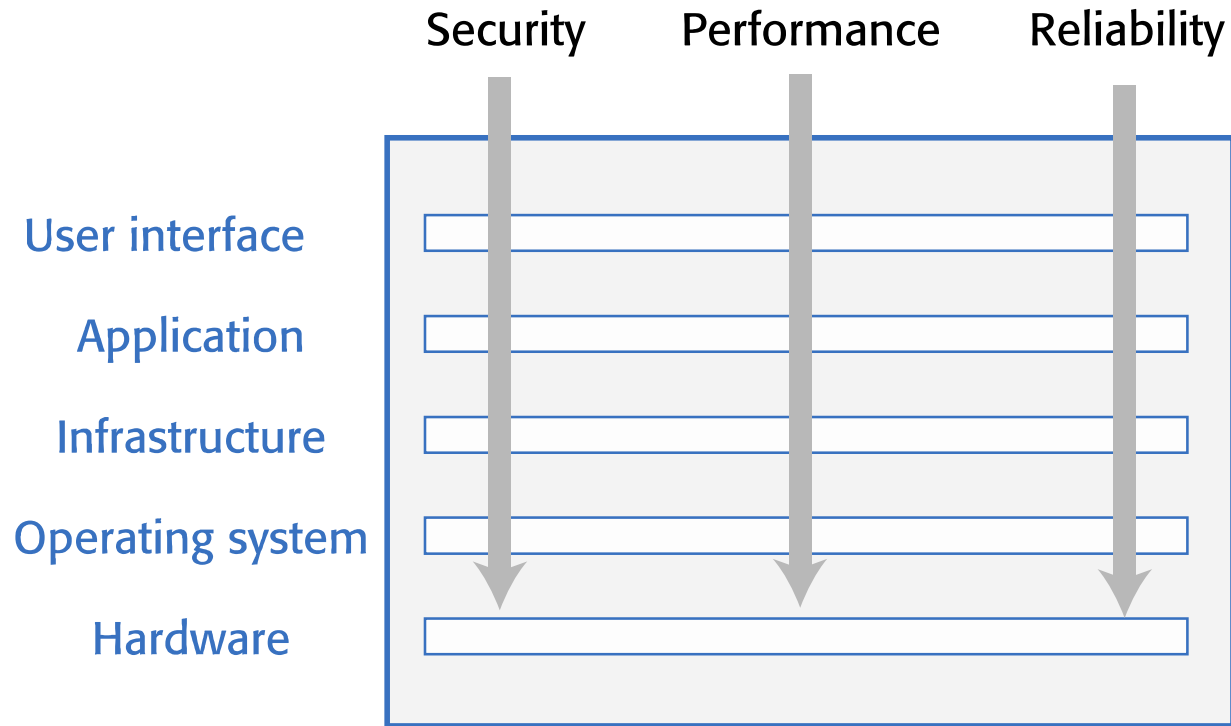
Design guidelines and layered architectures (2 of 2)

- The architectural model is a high-level model that does not include implementation information.
 - Ideally, components at level X (say) should only interact with the APIs of the components in level X-1. That is, interactions should be between layers and not across layers.

Cross-cutting concerns

- Cross-cutting concerns are concerns that are systemic, that is, they affect the whole system.
- In a layered architecture, cross-cutting concerns affect all layers in the system as well as the way in which people use the system.
- Cross-cutting concerns are completely different from the functional concerns represented by layers in a software architecture.
- Every layer has to take them into account and there are inevitably interactions between the layers because of these concerns.
- The existence of cross-cutting concerns is the reason why modifying a system after it has been designed to improve its security is often difficult.

Figure 4.9



Cross-cutting concerns

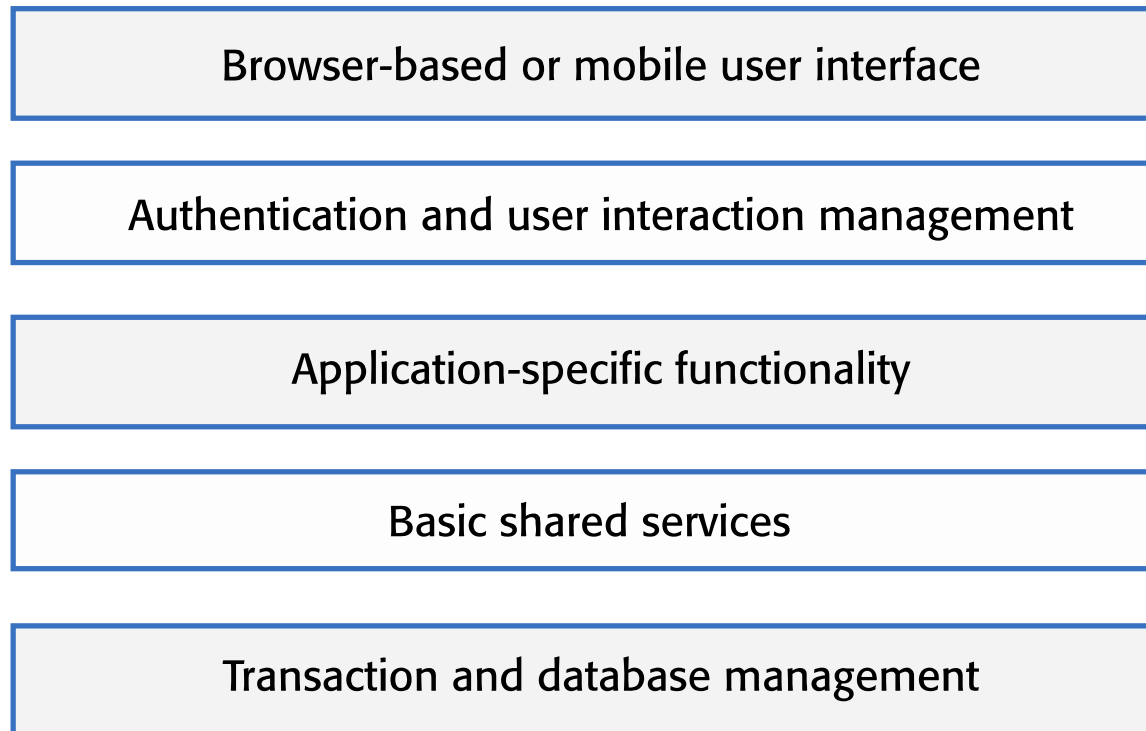
Table 4.5 Security as a cross-cutting concern

Security architecture

Different technologies are used in different layers, such as an SQL database or a Firefox browser. Attackers can try to use vulnerabilities in these technologies to gain access. Consequently, you need protection from attacks at each layer as well as protection at lower layers in the system from successful attacks that have occurred at higher-level layers.

If there is only a single security component in a system, this represents a critical system vulnerability. If all security checking goes through that component and it stops working properly or is compromised in an attack, then you have no reliable security in your system. By distributing security across the layers, your system is more resilient to attacks and software failure (remember the *Rogue One* example earlier in the chapter).

Figure 4.10



A generic layered architecture for a web-based application

Table 4.6 Layer functionality in a web-based application (1 of 2)

Layer	Explanation
Browser-based or mobile user interface	A web browser system interface in which HTML forms are often used to collect user input. Javascript components for local actions, such as input validation, should also be included at this level. Alternatively, a mobile interface may be implemented as an app.
Authentication and UI management	A user interface management layer that may include components for user authentication and web page generation.
Application-specific functionality	An “application” layer that provides functionality of the application. Sometimes this may be expanded into more than one layer.

Table 4.6 Layer functionality in a web-based application (2 of 2)

Layer	Explanation
Basic shared services	A shared services layer that includes components that provide services used by the application layer components.
Database and transaction management	A database layer that provides services such as transaction management and recovery. If your application does not use a database, then this may not be required.

Table 4.7 iLearn architectural design principles

Principle	Explanation
Replaceability	It should be possible for users to replace applications in the system with alternatives and to add new applications. Consequently, the list of applications included should not be hardwired into the system.
Extensibility	It should be possible for users or system administrators to create their own versions of the system, which may extend or limit the “standard” system.
Age-appropriate	Alternative user interfaces should be supported so that ageappropriate interfaces for students at different levels can be created.
Programmability	It should be easy for users to create their own applications by linking existing applications in the system.
Minimum work	Users who do not wish to change the system should not have to do extra work so that other users can make changes.

iLearn design principles

- Our goal in designing the iLearn system was to create an adaptable, universal system that could be easily updated as new learning tools became available.
 - This means that it must be possible to change and replace components and services in the system (principles (1) and (2)).
 - Because the potential system users spanned an age range from 3 to 18, we needed to provide age-appropriate user interfaces and to make it easy to choose an interface (principle (3)).
 - Principle (4) also contributes to system adaptability and principle (5) was included to ensure that this adaptability did not adversely affect users who did not require it.

Designing iLearn as a service-oriented system (1 of 2)

- These principles led us to an architectural design decision that the iLearn system should be service-oriented.
- Every component in the system is a service. Any service is potentially replaceable and new services can be created by combining existing services. Different services delivering comparable functionality can be provided for students of different ages.

Designing iLearn as a service-oriented system (2 of 2)

- Service integration
 - *Full integration* Services are aware of and can communicate with other services through their APIs.
 - *Partial integration* Services may share service components and databases but are not aware of and cannot communicate directly with other application services.
 - *Independent* These services do not use any shared system services or databases and they are unaware of any other services in the system. They can be replaced by any other comparable service.

Figure 4.11

User interface

Web browser	iLearn app
-------------	------------

User interface management

Interface creation	Forms management	Interface delivery	Login
--------------------	------------------	--------------------	-------

Configuration services

Group configuration	Application configuration	Security configuration	User interface configuration	Setup service
---------------------	---------------------------	------------------------	------------------------------	---------------

Application services

Archive access	Word processor	Video conf.	Email and messaging	User installed applications
Blog	Wiki	Spreadsheet	Presentation	Drawing

Integrated services

Resource discovery	User analytics	Virtual learning environment	Authentication and authorization
--------------------	----------------	------------------------------	----------------------------------

Shared infrastructure services

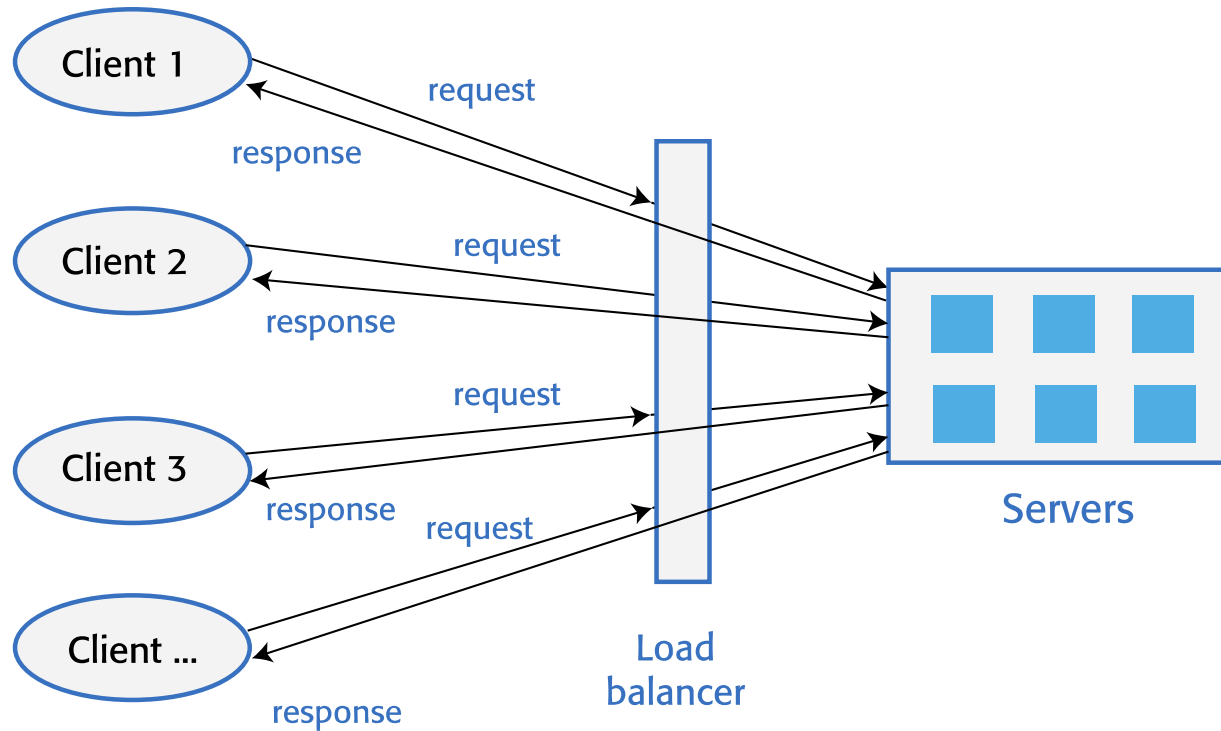
Authentication	Logging and monitoring	Application interfacing
User storage	Application storage	Search

A layered architectural model of the iLearn system

Distribution architecture

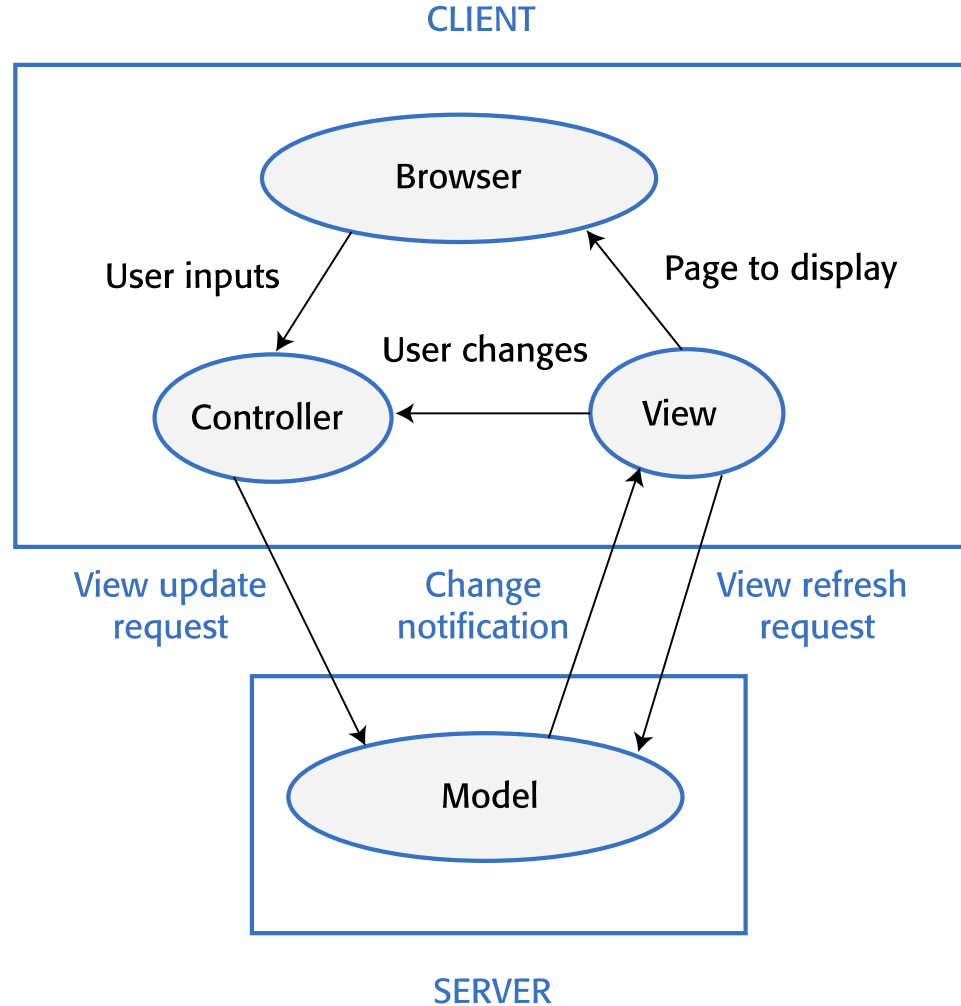
- The distribution architecture of a software system defines the servers in the system and the allocation of components to these servers.
- Client-server architectures are a type of distribution architecture that is suited to applications where clients access a shared database and business logic operations on that data.
- In this architecture, the user interface is implemented on the user's own computer or mobile device.
 - Functionality is distributed between the client and one or more server computers.

Figure 4.12



Client-server architecture

Figure 4.13



The Model-View-Controller pattern

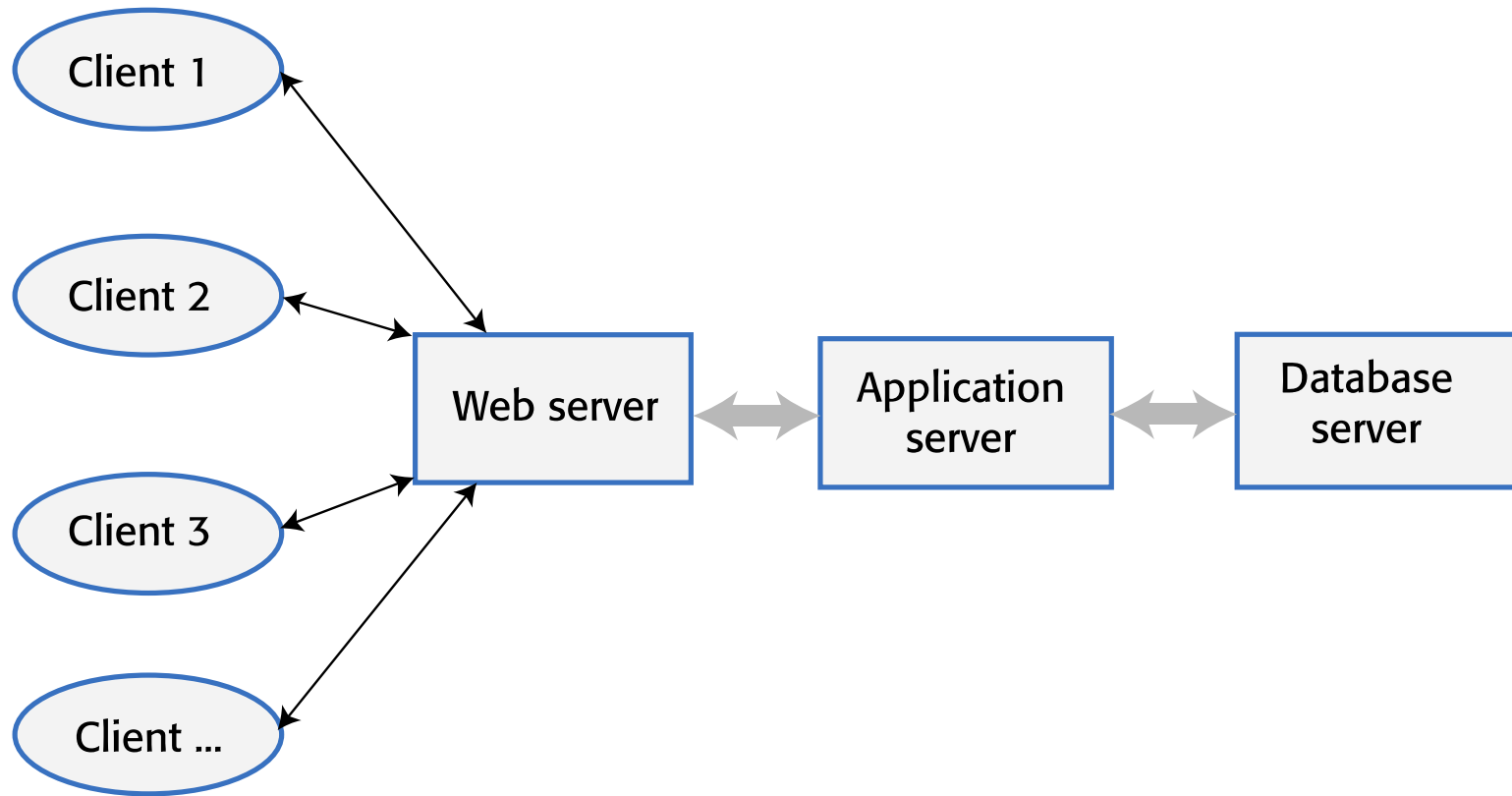
Client-server communication (1 of 2)

- Client-server communication normally uses the HTTP protocol.
 - The client sends a message to the server that includes an instruction such as GET or POST along with the identifier of a resource (usually a URL) on which that instruction should operate. The message may also include additional information, such as information collected from a form.

Client-server communication (2 of 2)

- HTTP is a text-only protocol so structured data has to be represented as text. There are two ways of representing this data that are widely used, namely XML and JSON.
 - XML is a markup language with tags used to identify each data item.
 - JSON is a simpler representation based on the representation of objects in the Javascript language.

Figure 4.14

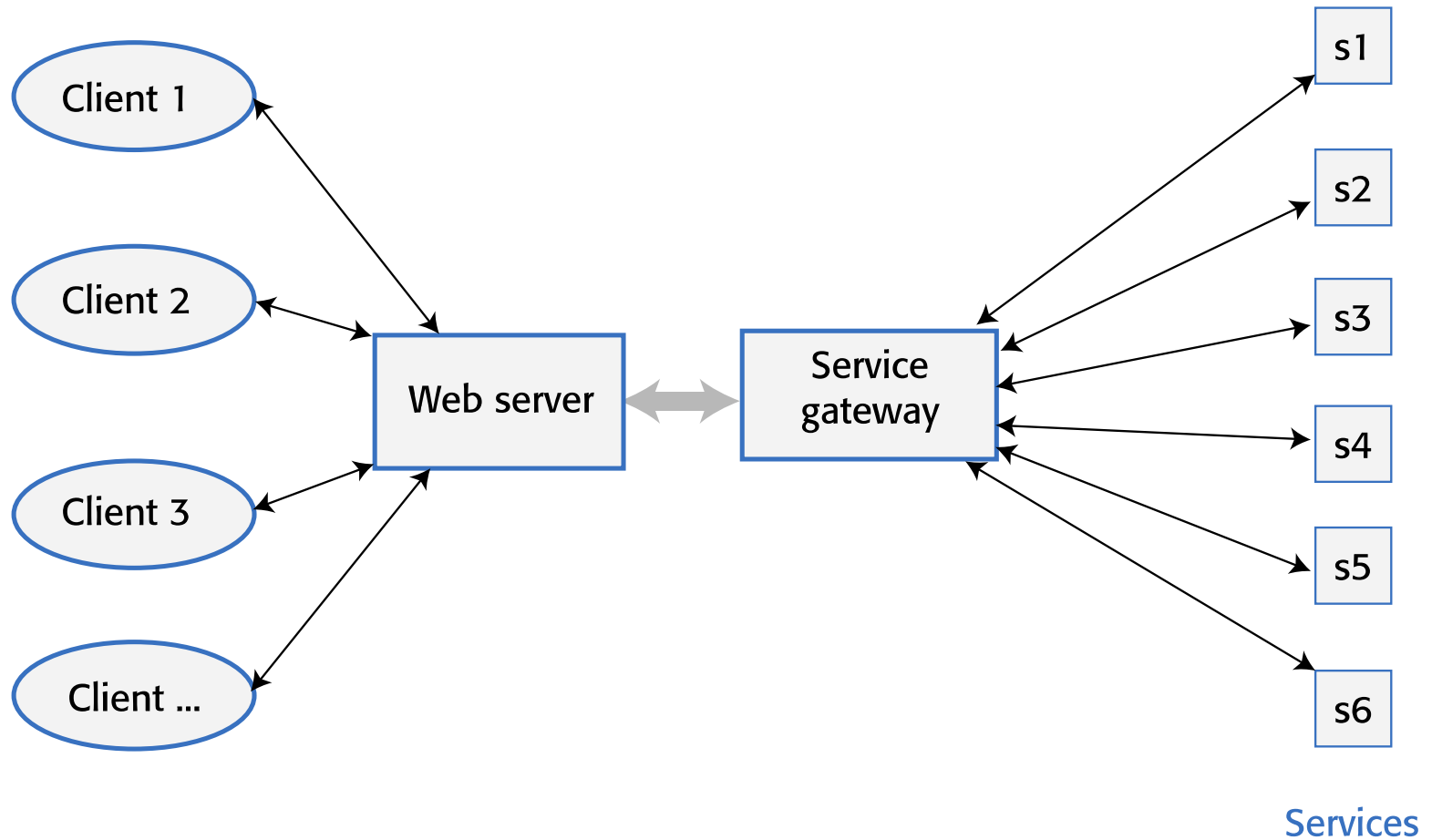


Multi-tier client-server architecture

Service-oriented architecture

- Services in a service-oriented architecture are stateless components, which means that they can be replicated and can migrate from one computer to another.
- Many servers may be involved in providing services
- A service-oriented architecture is usually easier to scale as demand increases and is resilient to failure.

Figure 4.15



Service-oriented architecture

Issues in architectural choice (1 of 2)

- Data type and data updates
 - If you are mostly using structured data that may be updated by different system features, it is usually best to have a single shared database that provides locking and transaction management. If data is distributed across services, you need a way to keep it consistent and this adds overhead to your system.
- Change frequency
 - If you anticipate that system components will be regularly changed or replaced, then isolating these components as separate services simplifies those changes

Issues in architectural choice (2 of 2)

- The system execution platform
 - If you plan to run your system on the cloud with users accessing it over the Internet, it is usually best to implement it as a service-oriented architecture because scaling the system is simpler.
 - If your product is a business system that runs on local servers, a multi-tier architecture may be more appropriate.

Table 4.8 Technology choices

Technology	Design decision
Database	Should you use a relational SQL database or an unstructured NoSQL database?
Platform	Should you deliver your product on a mobile app and/or a web platform?
Server	Should you use dedicated in-house servers or design your system to run on a public cloud? If a public cloud, should you use Amazon, Google, Microsoft, or some other option?
Open source	Are there suitable open-source components that you could incorporate into your products?
Development tools	Do your development tools embed architectural assumptions about the software being developed that limit your architectural choices?

Database (1 of 2)

- There are two kinds of database that are now commonly used:
 - Relational databases, where the data is organised into structured tables
 - NoSQL databases, in which the data has a more flexible, user-defined organization.
- Relational databases, such as MySQL, are particularly suitable for situations where you need transaction management and the data structures are predictable and fairly simple.

Database (2 of 2)

- NoSQL databases, such as MongoDB, are more flexible and potentially more efficient than relational databases for data analysis.
 - NoSQL databases allow data to be organized hierarchically rather than as flat tables and this allows for more efficient concurrent processing of 'big data'.

Delivery platform (1 of 3)

- Delivery can be as a web-based or a mobile product or both
- Mobile issues:
 - *Intermittent connectivity* You must be able to provide a limited service without network connectivity.
 - *Processor power* Mobile devices have less powerful processors, so you need to minimize computationally-intensive operations.

Delivery platform (2 of 3)

- Mobile issues:
 - *Power management* Mobile battery life is limited so you should try to minimize the power used by your application.
 - *On-screen keyboard* On-screen keyboards are slow and error-prone. You should minimize input using the screen keyboard to reduce user frustration.

Delivery platform (3 of 3)

- To deal with these differences, you usually need separate browser-based and mobile versions of your product front-end.
 - You may need a completely different decomposition architecture in these different versions to ensure that performance and other characteristics are maintained.

Server (1 of 2)

- A key decision that you have to make is whether to design your system to run on customer servers or to run on the cloud.
- For consumer products that are not simply mobile apps I think it almost always makes sense to develop for the cloud.

Server (2 of 2)

- For business products, it is a more difficult decision.
 - Some businesses are concerned about cloud security and prefer to run their systems on in-house servers. They may have a predictable pattern of system usage so there is less need to design your system to cope with large changes in demand.
- An important choice you have to make if you are running your software on the cloud is which cloud provider to use.

Open source (1 of 2)

- Open source software is software that is available freely, which you can change and modify as you wish.
 - The advantage is that you can reuse rather than implement new software, which reduces development costs and time to market.
 - The disadvantages of using open-source software is that you are constrained by that software and have no control over its evolution.

Open source (2 of 2)

- The decision on the use of open-source software also depends on the availability, maturity and continuing support of open source components.
- Open source license issues may impose constraints on how you use the software.
- Your choice of open source software should depend on the type of product that you are developing, your target market and the expertise of your development team.

Development tools (1 of 2)

- Development technologies, such as a mobile development toolkit or a web application framework, influence the architecture of your software.
 - These technologies have built-in assumptions about system architectures and you have to conform to these assumptions to use the development system.

Development tools (2 of 2)

- The development technology that you use may also have an indirect influence on the system architecture.
 - Developers usually favour architectural choices that use familiar technologies that they understand.
 - For example, if your team have a lot of experience of relational databases, they may argue for this instead of a NoSQL database.

Key points 1 (1 of 2)

- Software architecture is the fundamental organization of a system embodied in its components, their relationships to each other, and to the environment, and the principles guiding its design and evolution.
- The architecture of a software system has a significant influence on non-functional system properties such as reliability, efficiency and security.
- Architectural design involves understanding the issues that are critical for your product and creating system descriptions that shows components and their relationships.

Key points 1 (2 of 2)

- The principal role of architectural descriptions is to provide a basis for the development team to discuss the system organization. Informal architectural diagrams are effective in architectural description because they are fast and easy to draw and share.
- System decomposition involves analyzing architectural components and representing them as a set of finer-grain components.

Key points 2 (1 of 2)

- To minimize complexity, you should separate concerns, avoid functional duplication and focus on component interfaces.
- Web-based systems often have a common layered structure including user interface layers, application-specific layers and a database layer.
- The distribution architecture in a system defines the organization of the servers in that system and the allocation of components to these servers.

Key points 2 (2 of 2)

- Multi-tier client-server and service-oriented architectures are the most commonly used architectures for web-based systems.
- Making decisions on technologies such as database and cloud technologies are an important part of the architectural design process.

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

CS673 Software Engineering

Software Design Part II
Patterns

In Design, We Allocate Responsibilities

- The object-oriented approach to software design is about allocating responsibilities (data, methods) to different classes in the program
 - and to different team members working on different parts of the program
- It is important to keep people's focus of attention as narrow as possible
 - so that they do not keep in mind extra details about other parts of the project

Design Has To Be Communicated

- This means that the developers must be able to describe and discuss their solutions and compare alternatives
 - in a language to be easily understood
- This language should be able to describe each component and its responsibilities
 - and relationships among components
- Design specification is one solution: text
 - it describes data fields, methods, rules of their use, collaborating classes, inheritance

Design Has To Be Communicated

- Textual description is adequate when it describes the properties of one piece
 - however, it is not informative enough to describe relationships between pieces
 - it takes time to write it down in all details and check for consistency in many places
 - it is not pliable enough to be easily changed when different design alternatives are discussed and compared
- Since it specifies all details at the same level, it cannot be used for an overview

UML

Unified Modeling Language

- UML: the Unified Modeling Language
 - diagrammatic representation (model) of relationships among different sets of program entities
- These are several languages used at different stages of development
 - analysis: Case Diagrams (interactions with external entities and their primary steps) and Activity Diagrams (steps of workflow of the program domain)

Unified Modeling Language

- Some other kinds of UML diagrams:
 - design: Class Diagrams (class name, attributes, behavior, relationships) and State Diagrams (possible object states and possible transitions between these states)
 - detailed design: several kinds of Interaction Diagrams (details of how objects interact with each other), e.g., Sequence Diagrams
 - deployment: Deployment Diagrams that show how modules interact when deployed
- It is a huge set of complex languages (hard to learn, v2.5.1 spec is 750 pages)

<https://www.omg.org/spec/UML/>

Unified Modeling Language

- Another way to classify UML diagrams:
 - Structure Diagrams: Class Diagram, Object Diagram, Component Diagram, Composite Diagram, Package Diagram, Deployment Diagram
 - Behavior Diagrams: Use Case Diagram, Activity Diagram, State Machine Diagram
 - Interaction Diagrams: Sequence Diagram, Communication Diagram, Timing Diagram, Interaction Overview Diagram

UML as a Communication Tool

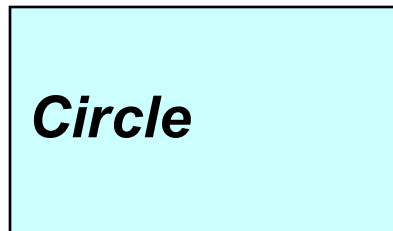
- The UML is first and foremost a communication tool
 - between members of development team
 - between the team and its customers (if customers learn this language) – to solicit better and more complete requirements
 - or even for an individual developer in the process of evaluating design alternatives
- It is easy to overuse the UML
 - and waste a lot of time drawing diagrams

UML as a Communication Tool

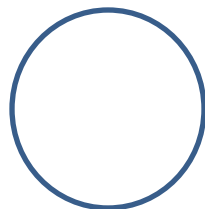
- Used properly, the UML facilitates communications
 - it helps to determine whether all participants have the same understanding of the system and its components
- It allows them to discuss the system at different levels of detail
 - from detailed diagrams showing many implementation-related details to high-level overview of the system in general

Class Diagram

- The most basic UML diagram is the Class Diagram
 - it describes classes that are part of the system (or components of some part of the system)
 - and it shows the relationships among classes and among objects of these classes

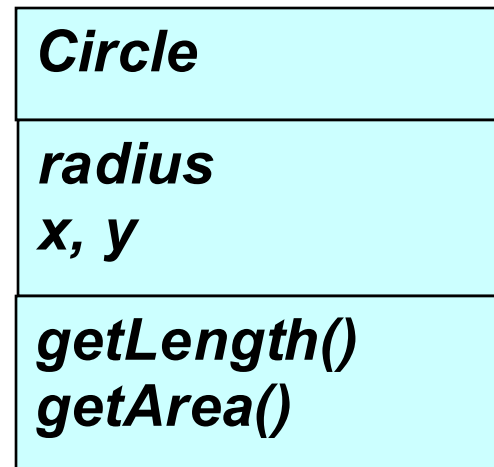
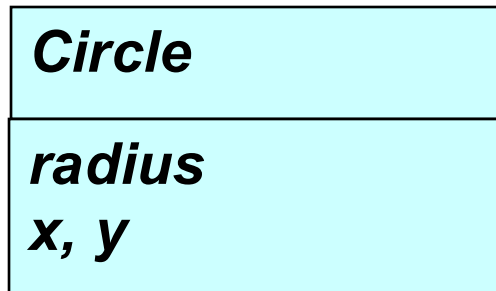


The most basic form of the class diagram contains rectangles with minimum information: just the class name



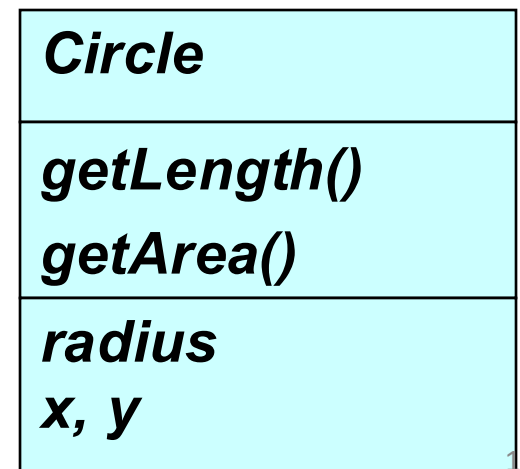
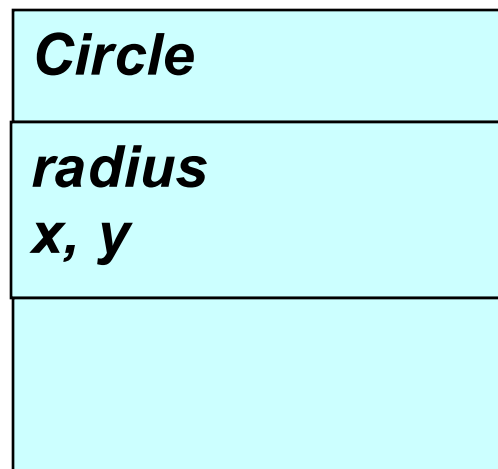
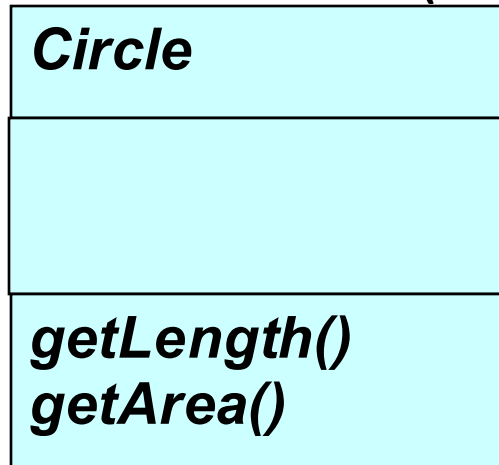
Class Diagram

- A more detailed notation shows design decisions related to
 - data fields of the class
 - the methods of the class
- The rectangle is divided into two or three regions to show data and methods



Class Diagram

- Sometimes the designer is interested in data only or in behavior only
 - it is ok to use a rectangle with three sections but fill only one or two of them
 - the OMT notation swapped the data and behavior sections (GoF use this notation)



Class Diagram

- UML allows the designer to describe the visibility of components (too detailed?)
 - a prefix '-' describes a private component, '+' denotes public, a '#' means protected

Circle

-radius: int
-x, y: int

#circleDisplay(): String
+getLength(): double
+getArea(): double
+toString(): String
+setRadius(r: int): void

In the type notation, the type follows the object; this is the Pascal/Ada convention rather than C/C++ (and reflects the OMG preferences)

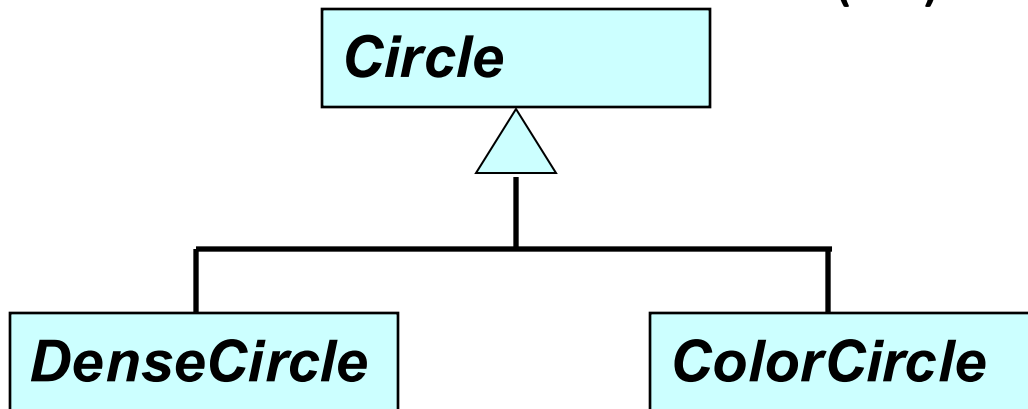
Class Diagram

- The more details we put into the class diagram, the more it approaches the design specification (plain text is better)
 - the less space is left on the page to describe relationships among classes
- When several classes participate in the design under discussion, most designers use just the class name on diagrams describing relationships
 - relationships are represented by drawing lines among squares representing classes

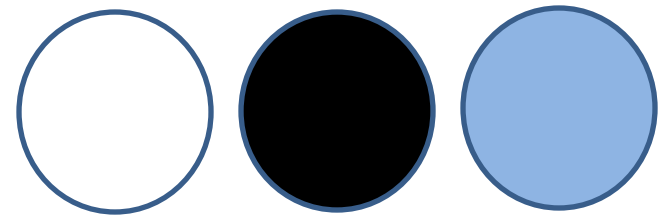
Object Oriented Design

Inheritance Relationship

- The 'is a' inheritance relationship between a superclass and a subclass: an arrow with a hollow arrowhead
 - the arrowhead might be for each subclass or one for all of them; the superclass these days is always drawn above subclass(es)

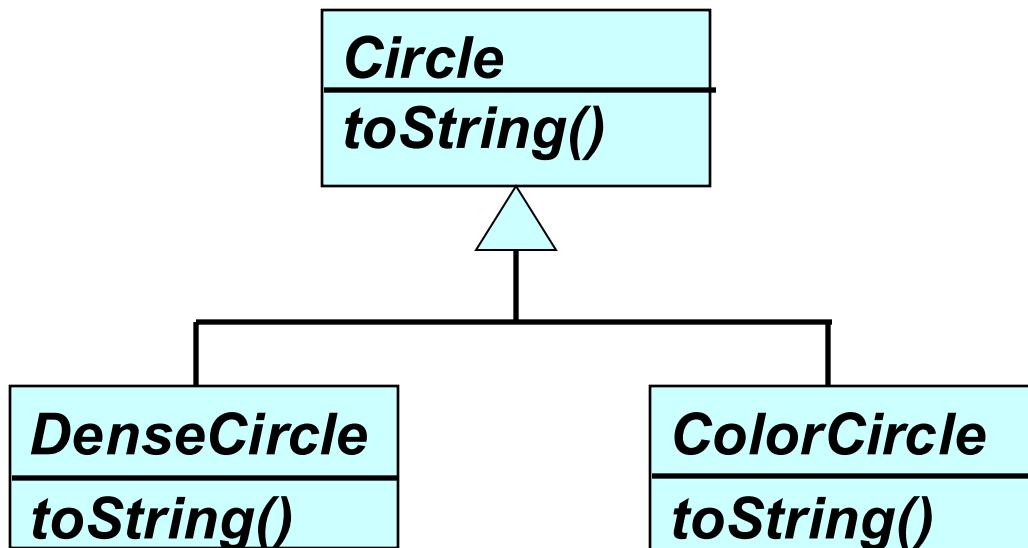


Here, classes **DenseCircle** and **ColorCircle** extend class **Circle**



Inheritance Relationship

- When inheritance is used for specification, the same notation is used
 - the abstract class name is italicized
 - so is the name of abstract methods that are implemented by subclasses

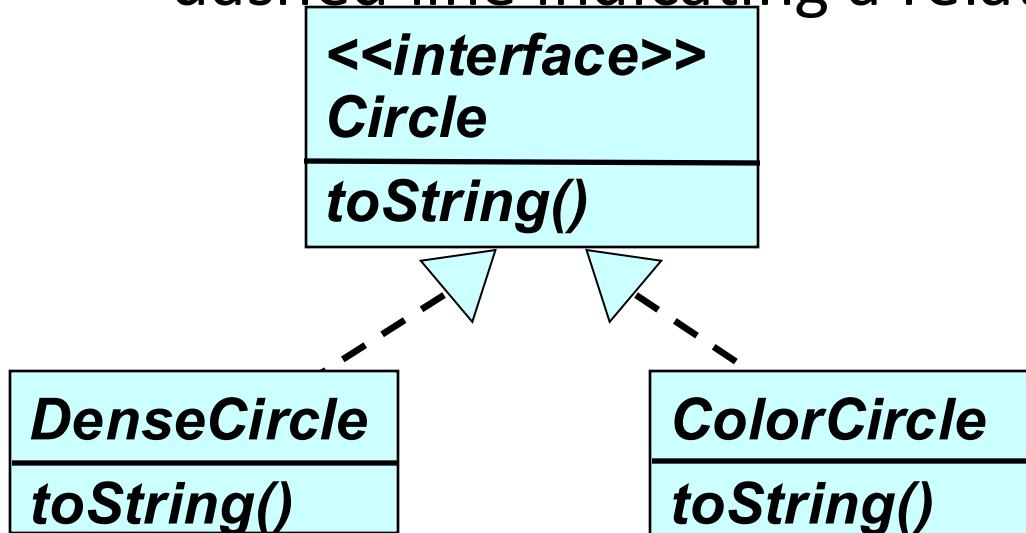


Here, class *Circle* is an abstract class

- the data section is not included
- subclasses have *toString()*, too

Implementing Relationship

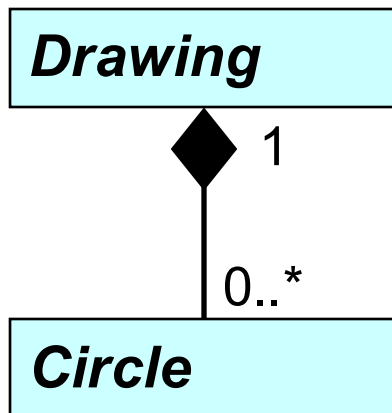
- When the superclass is an interface rather than an abstract class
 - then "subclasses" implement the interface rather than extend the superclass
 - the notation is similar: same arrowhead but a dashed line indicating a relationship



Here, DenseCircle and ColorCircle implement the Circle interface

Composition Relationship

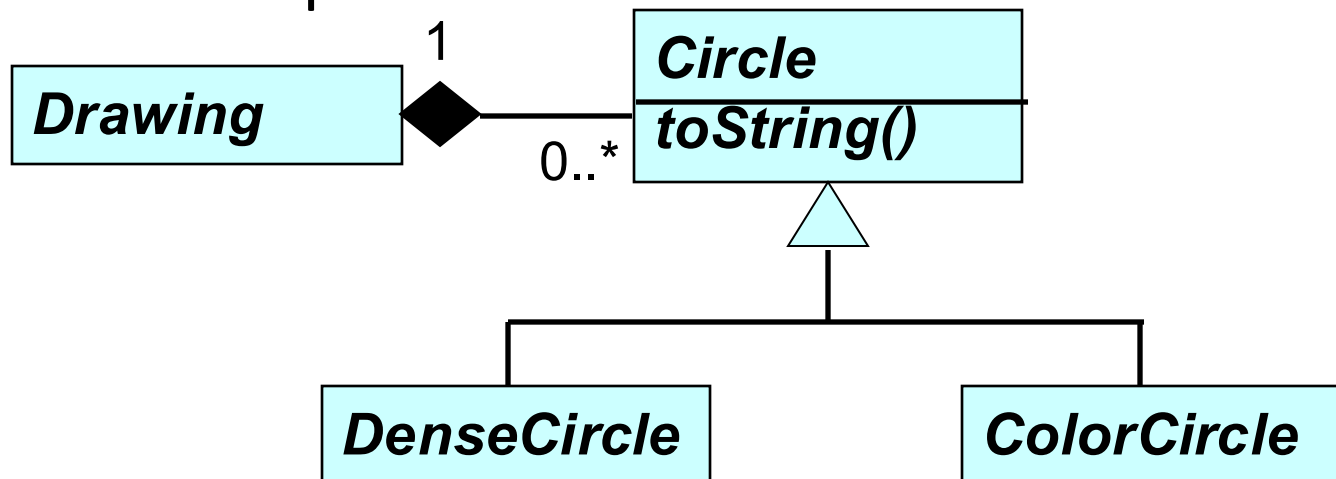
- The 'has a' relationship is used when object of one class has an object of another class as its component
 - the component objects are created when the container (composite) object is created (or some time later); they disappear when the container object disappears



The cardinality notation describes how many objects can participate in the relationship (e.g., 0..n)

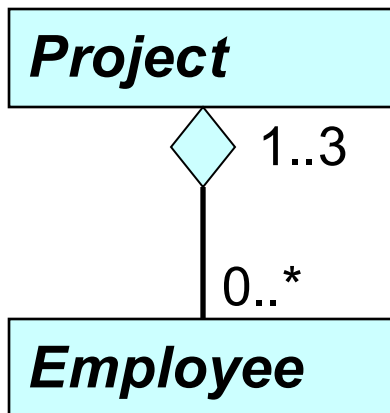
Composition Relationship

- If a component object belongs to the inheritance hierarchy, the whole hierarchy is displayed as a component
 - attention: if the superclass of the hierarchy is an abstract class, only subclass objects can become actual components!



Aggregation Relationship

- The UML calls this relationship between objects (not classes!) composition
 - components exists in the container and nowhere else, not shared with others
- If a component can exist independently of the container, it is called aggregation



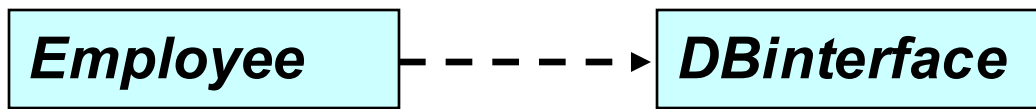
The hollow diamond says that components might be created independently of the container

Aggregation Relationship

- You can view composition as an unshared (exclusive) relationship between the container and components
 - aggregation means that the container can share components with other containers
 - the life cycle of component is not tied to the life cycle of the container (non-exclusive)
- GoF use the reverse terminology (OMT)
 - composition means a shared relationship
 - aggregation means exclusive use
 - their work precedes UML by several years

General Association

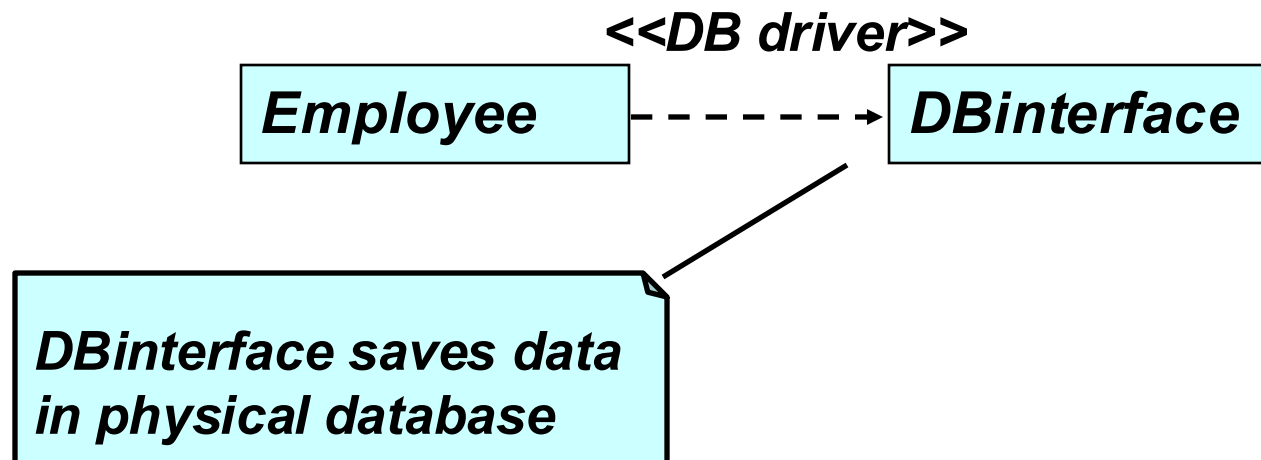
- An unspecified association between classes is denoted by a solid line
- A directional association is denoted by a dashed (or dotted) line with an arrow
 - one object uses another, or depends on another or creates another



Here, DBinterface saves
Employee data in the database

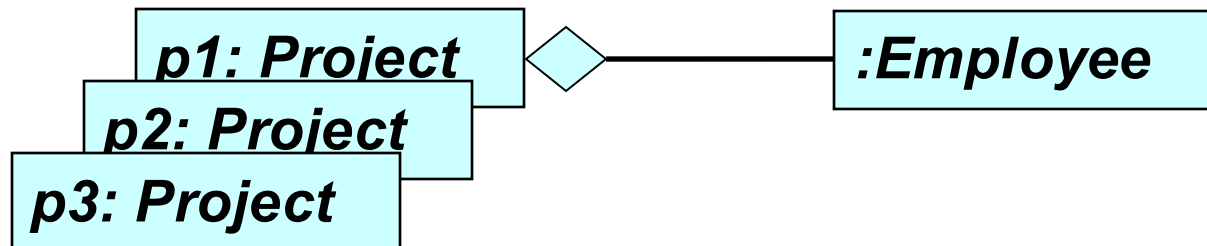
Stereotypes and Notes

- The existence of dependency relationship reflects the existence of a semantic association among objects
 - to specify its meaning we use "stereotypes" (text in double angle brackets) or notes (text in a box with the right corner folded)



Object Diagram

- Class diagrams show static relationships between classes
 - they do not show any activities
- Object diagrams show how many objects of each class exist at a particular moment during execution
 - They include object name and its type or type only (preceded by a semicolon)

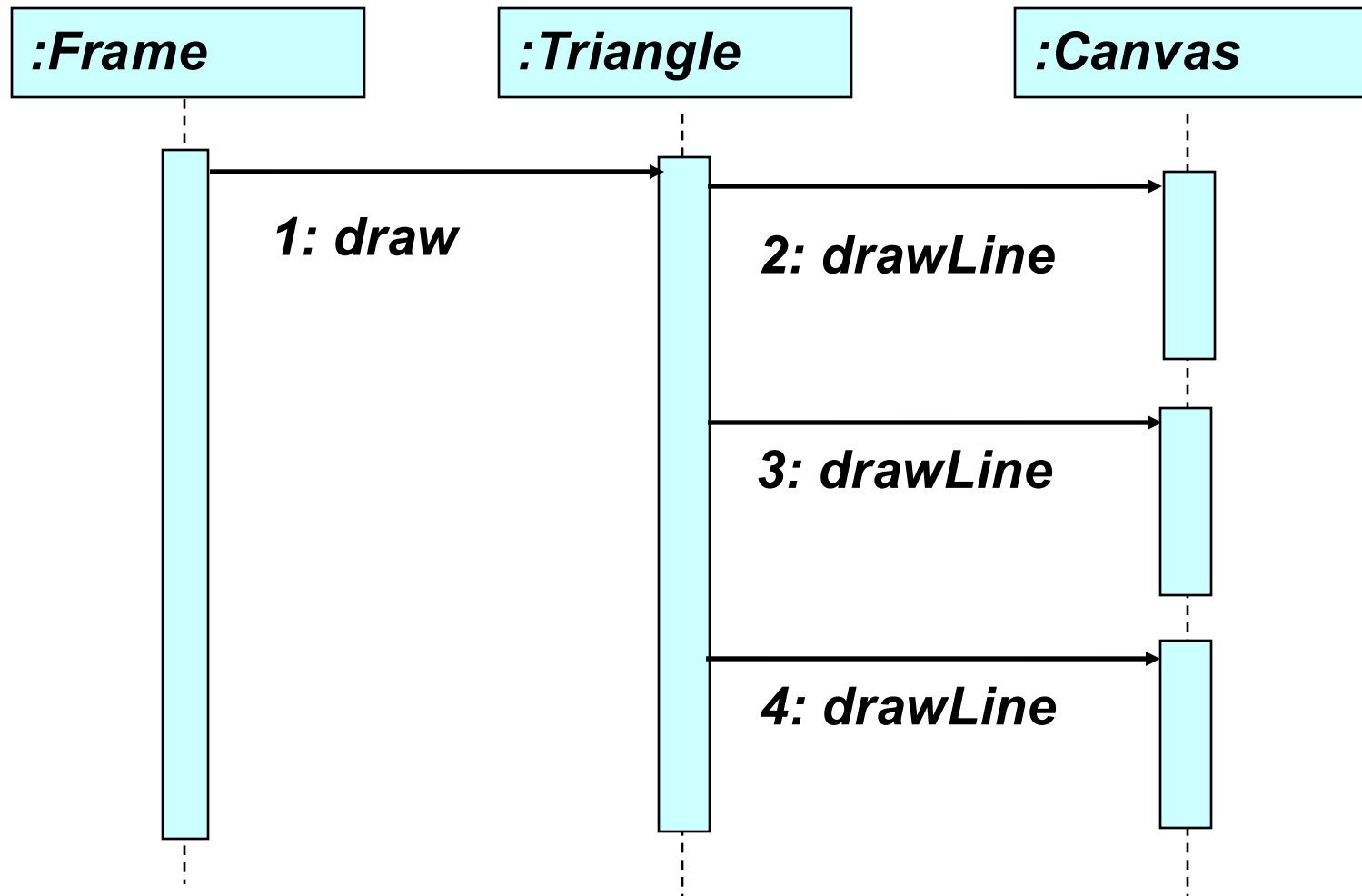


Interaction Diagram

- Another technique for showing actions between objects is using one of the Interaction Diagrams
 - the Sequence Diagram is the most common type of Interaction Diagram
- They are read from top to bottom
 - rectangles on top represent objects
 - same naming convention as for the object diagram: either 'object : type' or just ':type'
 - the vertical lines represent the lifeline of the objects: time goes from top down

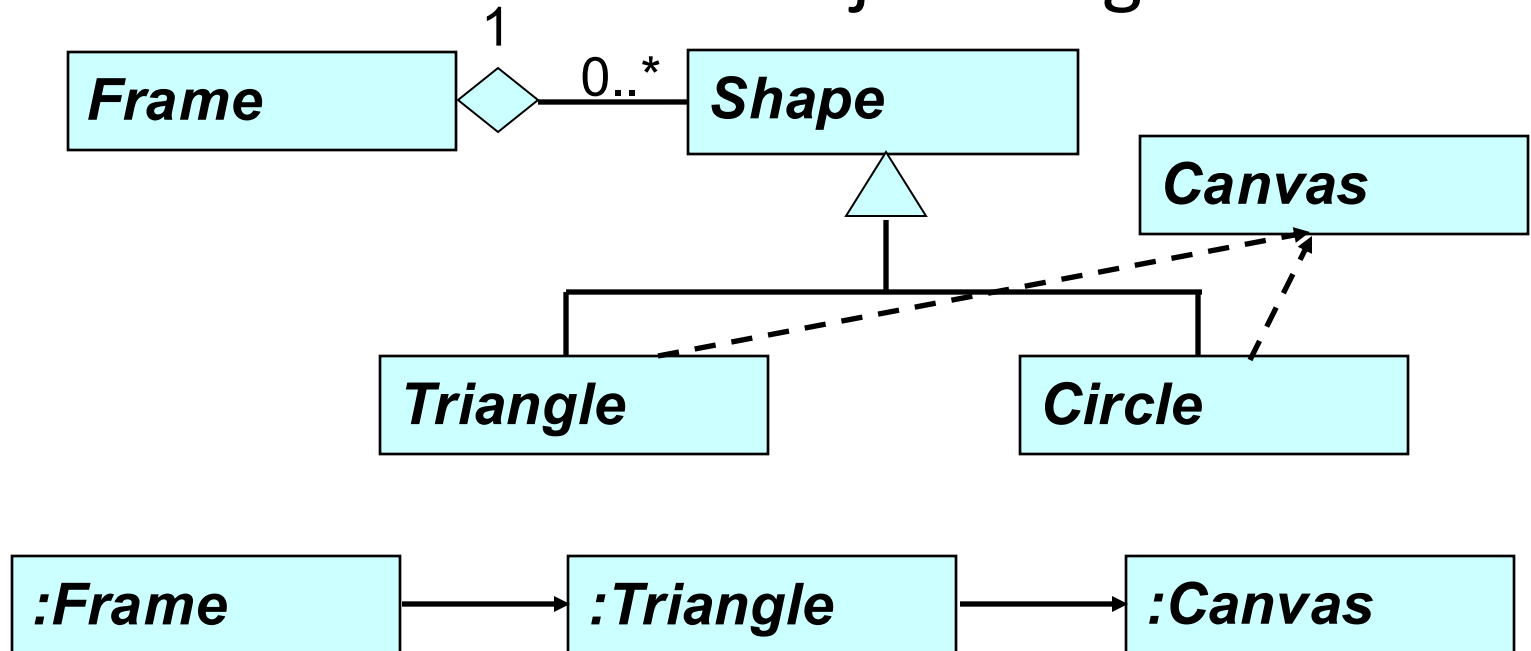
Sequence Diagram

- Horizontal lines represent method calls



Sequence Diagram

- As appropriate for the sequence diagram, it represents the sequence of actions of interest during execution
- Here are a class and an object diagram



Design Patterns

Inheritance as a Design Pattern

- Inheritance is actually a design pattern
 - it is often used alone as a powerful design technique (allocation of responsibilities)
 - it is used in combination with other design patterns (as the major part of design)
- It supports two goals of the object-oriented approach
 - encapsulation of what we hope will be permanent (in the superclass)
 - encapsulation of what we think will change (in subclasses)

Inheritance as a Design Pattern

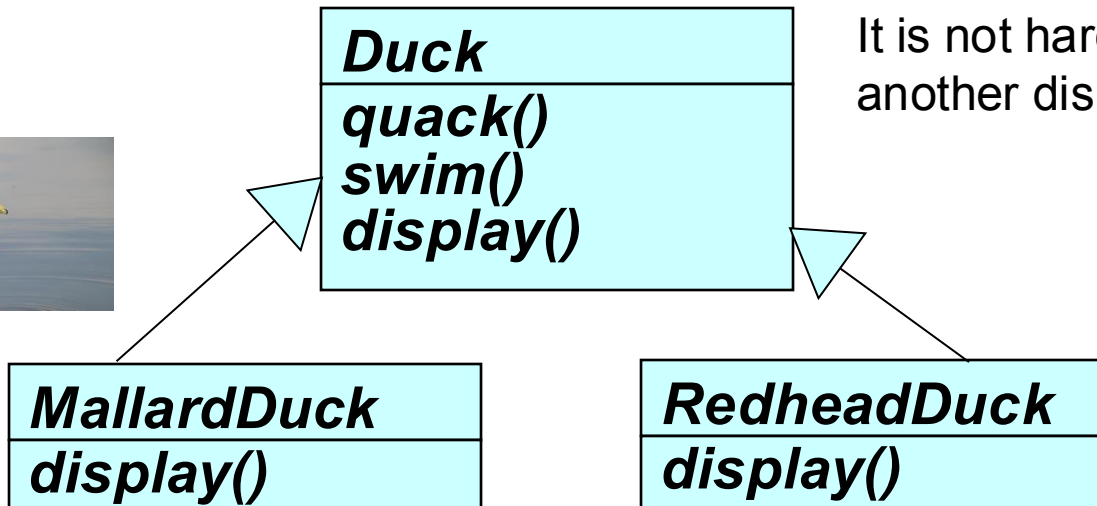
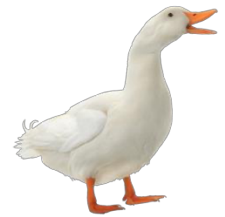
- The variable part usually encompasses different modes of behavior
 - each subclass encapsulates its separate mode of behavior, e.g., `getPay()`
 - they are different for each subclass; still, we use the same method name in all subclasses since they are similar enough
 - when the system has to support yet another mode of behavior, we add another subclass with the same method names
 - this is just beautiful!

Inheritance as a Design Pattern

- When what is expected to stay permanent changes nonetheless
 - it often introduces a new pattern of behavior common to all subclasses (e.g., method `changeName()`)
 - we add it to the superclass and this behavior is automatically propagated to all subclasses (inheritance for extension)
 - this is also beautiful!
 - but we have a problem when this behavior cannot be applied to all subclasses

An Example: a Simple Inheritance

- Example: a duck pond simulation game
 - common behavior includes `quack()`, `swim()` and `display()`
 - however, different kinds of ducks display differently on the screen;
 - still, these differences are small enough

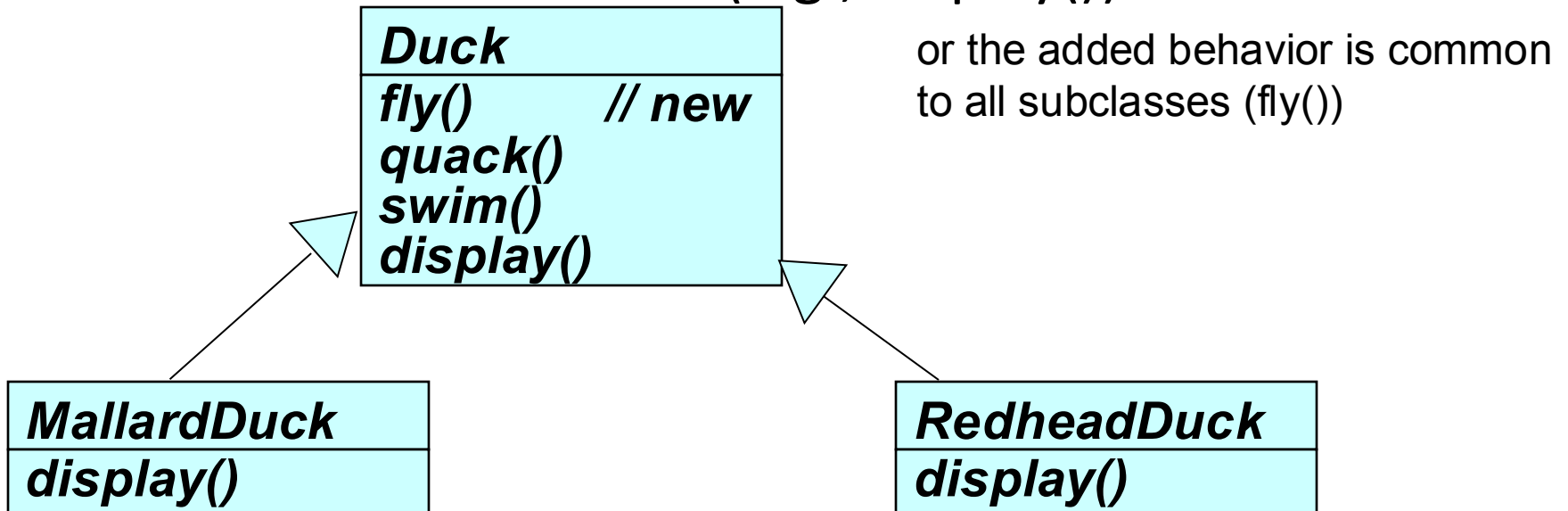


It is not hard to add a duck with another `display()` behavior



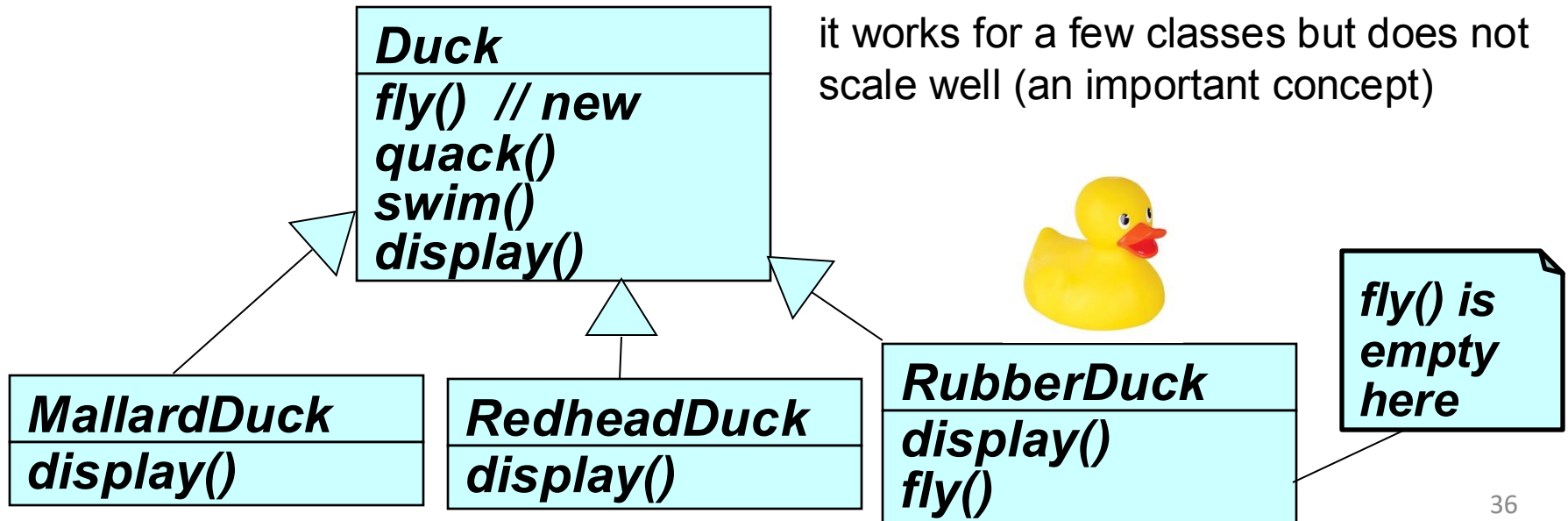
An Example: a Simple Inheritance

- If we need to add behavior common to all kinds of objects, we add it to superclass
 - indeed, inheritance is a great tool for reuse
 - as long as behavior varies little so that the same name can be used (e.g., `display()`)



An Example: a Simple Inheritance

- If a new Duck subclass adds a new dimension of variance (e.g., it cannot fly)
 - the use of inheritance becomes awkward
- We can override fly() in the subclass as an empty method: this is ok but very ugly

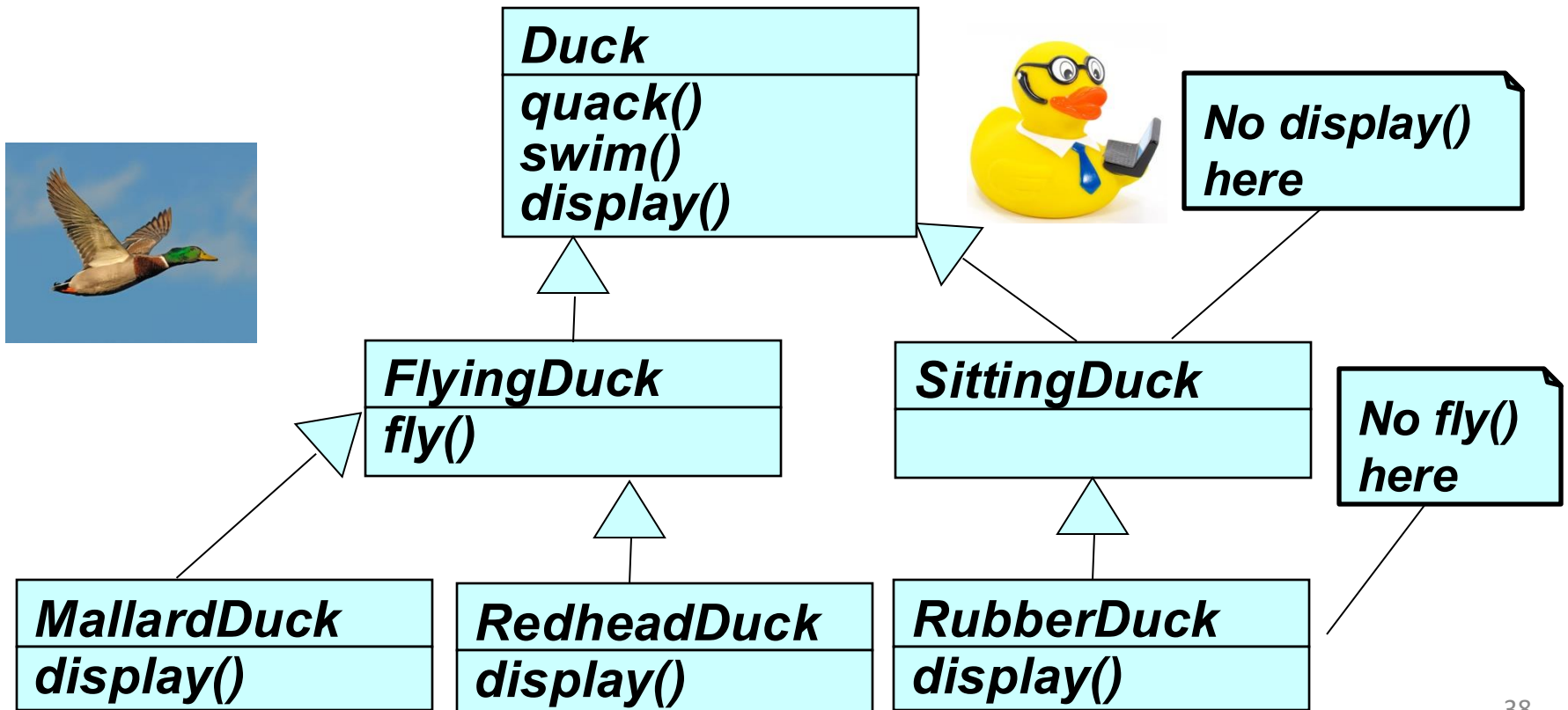


Limitations of Inheritance

- If behavior varies among all subtypes but remains essentially the similar
 - inheritance is a sufficient design pattern
- If behavior applies to some subclasses but not to all, inheritance leads to using a large number of empty methods
 - the number of aspects of variance and exceptions can become large (some ducks do not fly, others do not quack, and so on)
- As a community, we use inheritance too much; this results in bad designs

Limitations of Inheritance

- Another way: make the hierarchy taller
 - encapsulate the new aspect in additional levels of the hierarchy

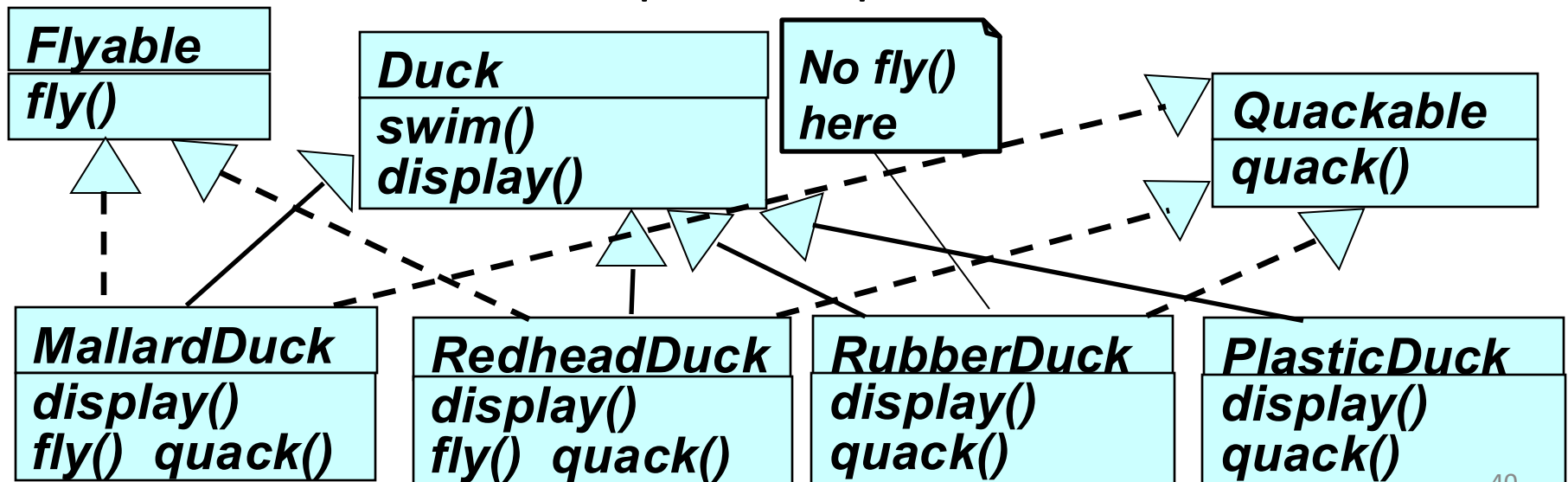


Limitations of Inheritance

- So, adding new class RubberDuck requires changes in all classes in the inheritance hierarchy and adding more classes
 - this is doable, but scalability is poor
- If we need to vary yet another aspect, e.g., the way the ducks quack (some do not, some squeak) we override this method in each subclass
 - if the behavior deserves another name (e.g., squeak instead of quack) we add yet another intermediate level of inheritance
 - and what if a class has to belong to two sub-hierarchies (it squeaks but flies)?

Limitations of Inheritance

- One possible approach to the growth of empty overrides is the use of interfaces
 - since not all ducks quack or fly, we can define interfaces, e.g., Flyable, Quackable
 - those ducks that fly implement Flyable
 - the ducks that quack implement Quackable



Limitations of Inheritance

- This is a nice idea, but it leads to bad design: duplicate code and multiple changes during maintenance
 - ducks that fly in the same way duplicate fly behavior (both data and method code)
 - changes to this behavior require changes in each subclass that implements fly()
- Interface is a mechanism of coercion
 - its use forces the subclass to implement a method, but it results in duplication of data and code across subclasses (poor reuse)

Encapsulate All That Vary

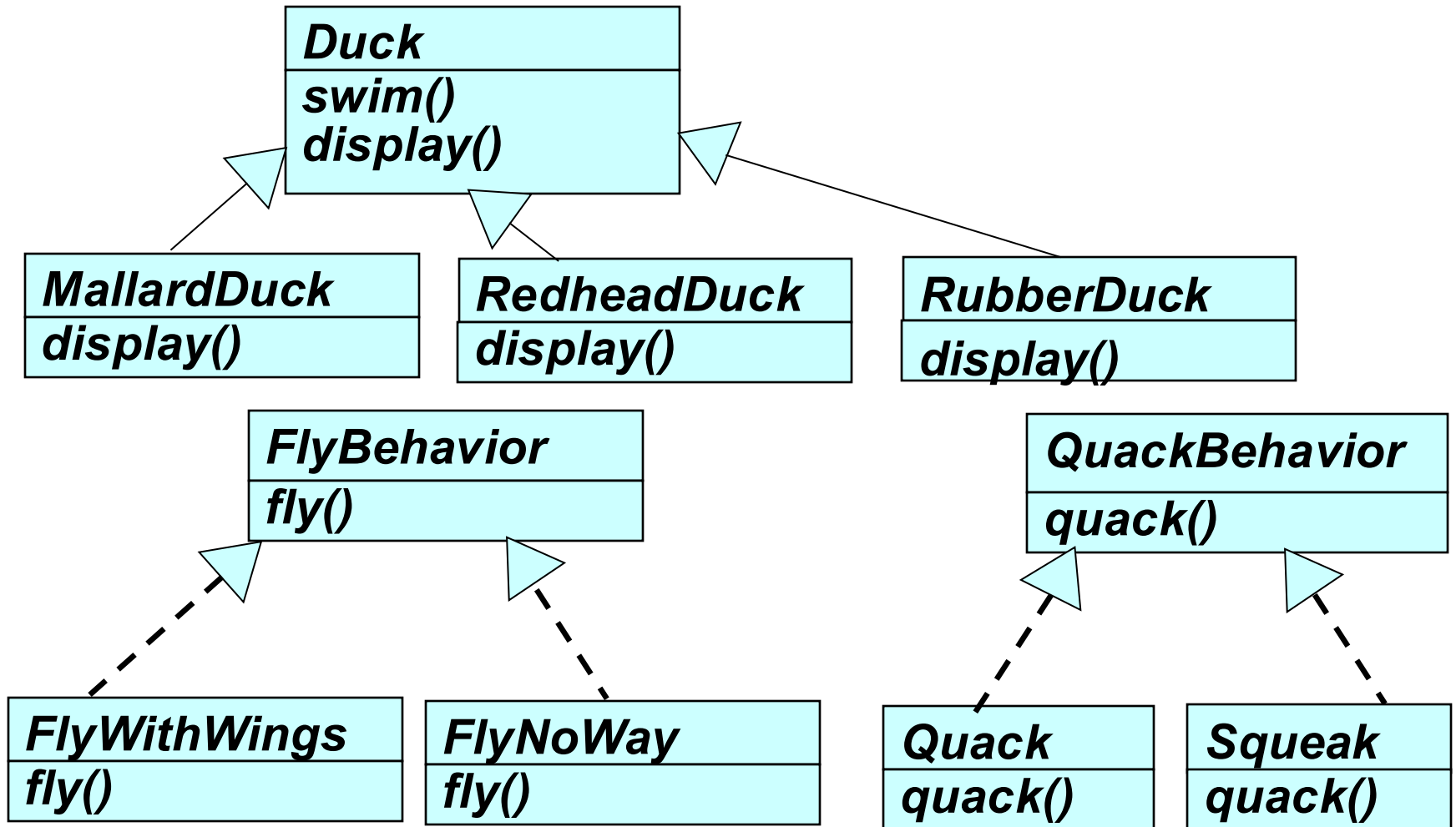
- Design principle: identify the aspects of the application that vary and separate them from what stays the same
 - encapsulate it so that it could change without affecting other parts of design
 - another form of a) info hiding, b) separation of concerns, c) using separate abstraction levels, d) pushing work down to servers, or e) low coupling: a more flexible system
- Ultimately, everything can change
 - but some things are more stable and others change relatively soon (subjective)

Encapsulate All That Vary

- Inheritance is still a great design technique: it encapsulates variability
 - but each inheritance hierarchy should encapsulate one aspect of variability only
- Our trouble stems from an attempt to add two aspects to the same hierarchy
 - flying behavior and quacking behavior
- Instead, let us create a hierarchy for each aspect of variability
 - flying behavior with its subclasses
 - quacking behavior with its subclasses

One Varying Aspect per Hierarchy

- Here, each aspect has its own hierarchy



One Varying Aspect per Hierarchy

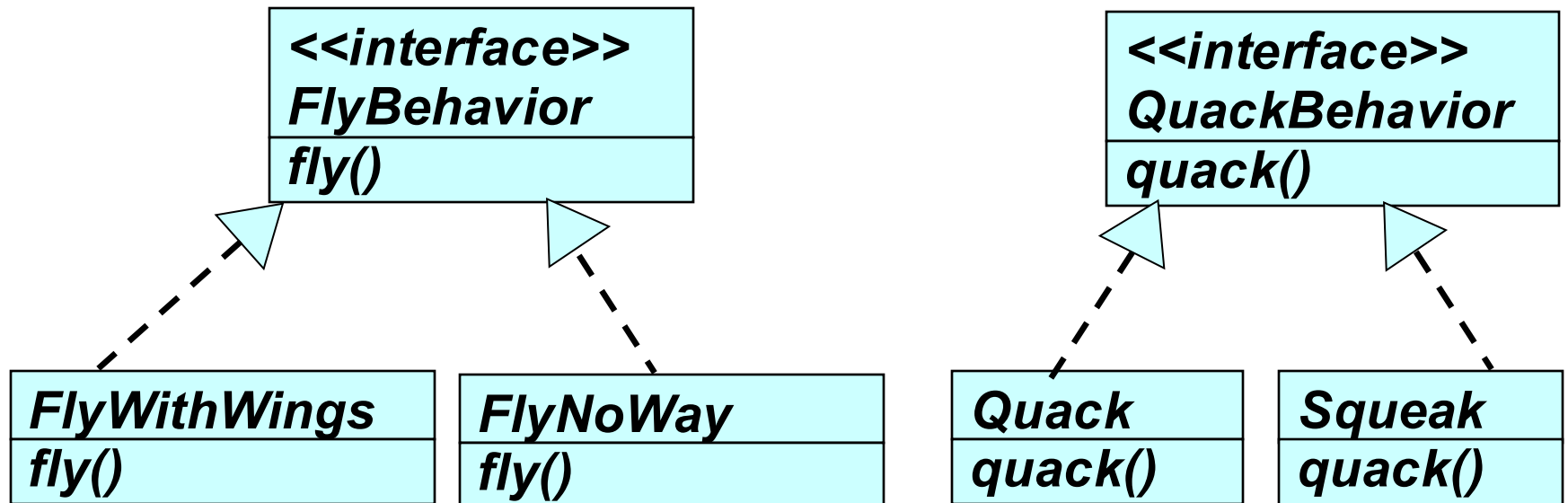
- Now methods `fly()` and `quack()` should be eliminated from the Duck superclass
 - not all Duck subclasses should inherit these methods: some will override them in their own way, some do not have them
- Instead, we create two other hierarchies
 - the flying behavior hierarchy will have subclasses with method `fly()` for flying with winds, flying like a rocket, or not flying at all
 - the quacking behavior hierarchy will have subclasses for quacking, squeaking, etc.

One Varying Aspect per Hierarchy

- One of side benefits will be the ability to change object behavior at run time
 - actually, we did not ask for this flexibility
 - this will impose on the client (simulation program) an additional job to specify behavior even when it is stable and can be specified in server classes (duck objects) themselves
- We use here the important design principle: program to an interface, not to implementation, use superclass pointers

One Varying Aspect per Hierarchy

- In this new design, superclasses only specify behavior (as interfaces or as abstract classes)
 - actual specific behavior is implemented by subclasses that implement the interfaces

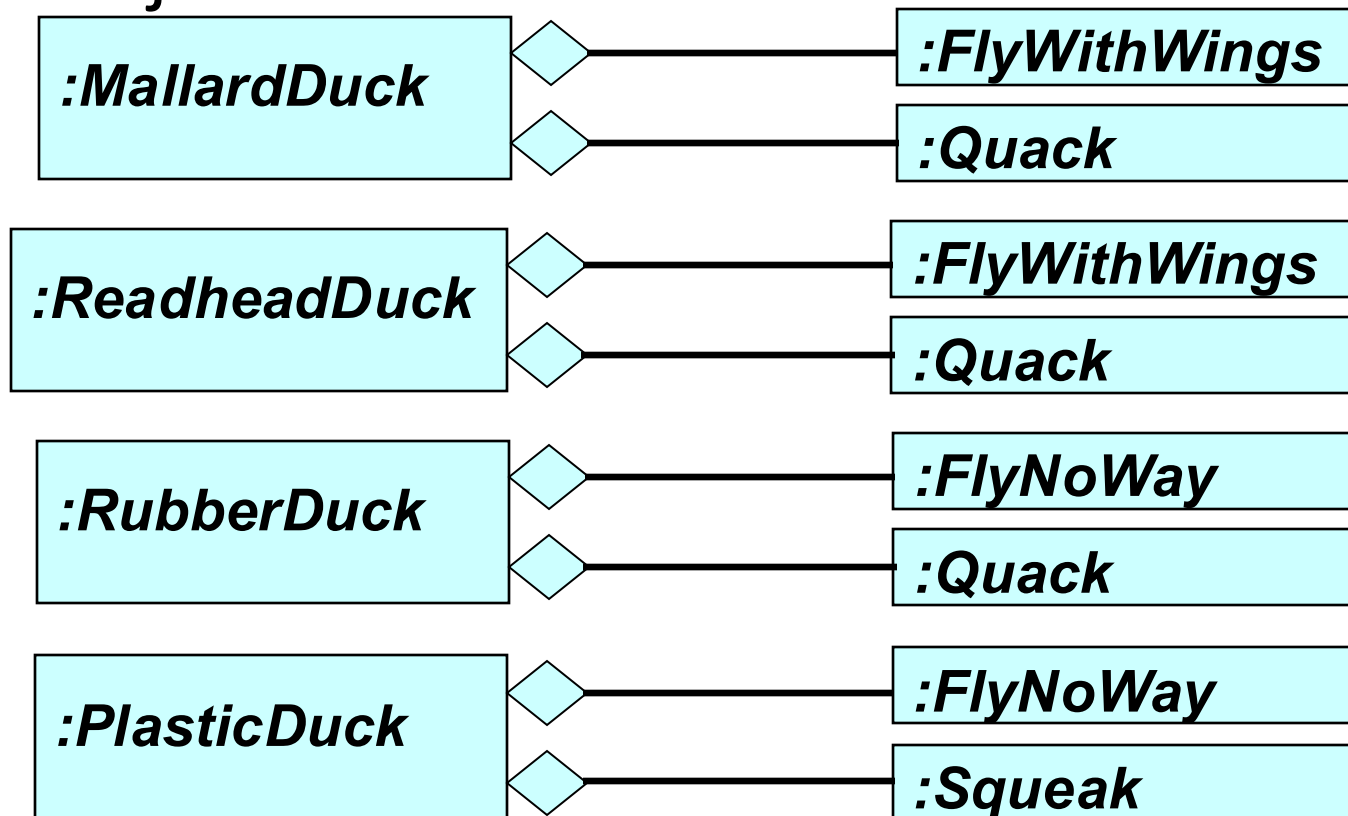


Link Hierarchies with Aggregation

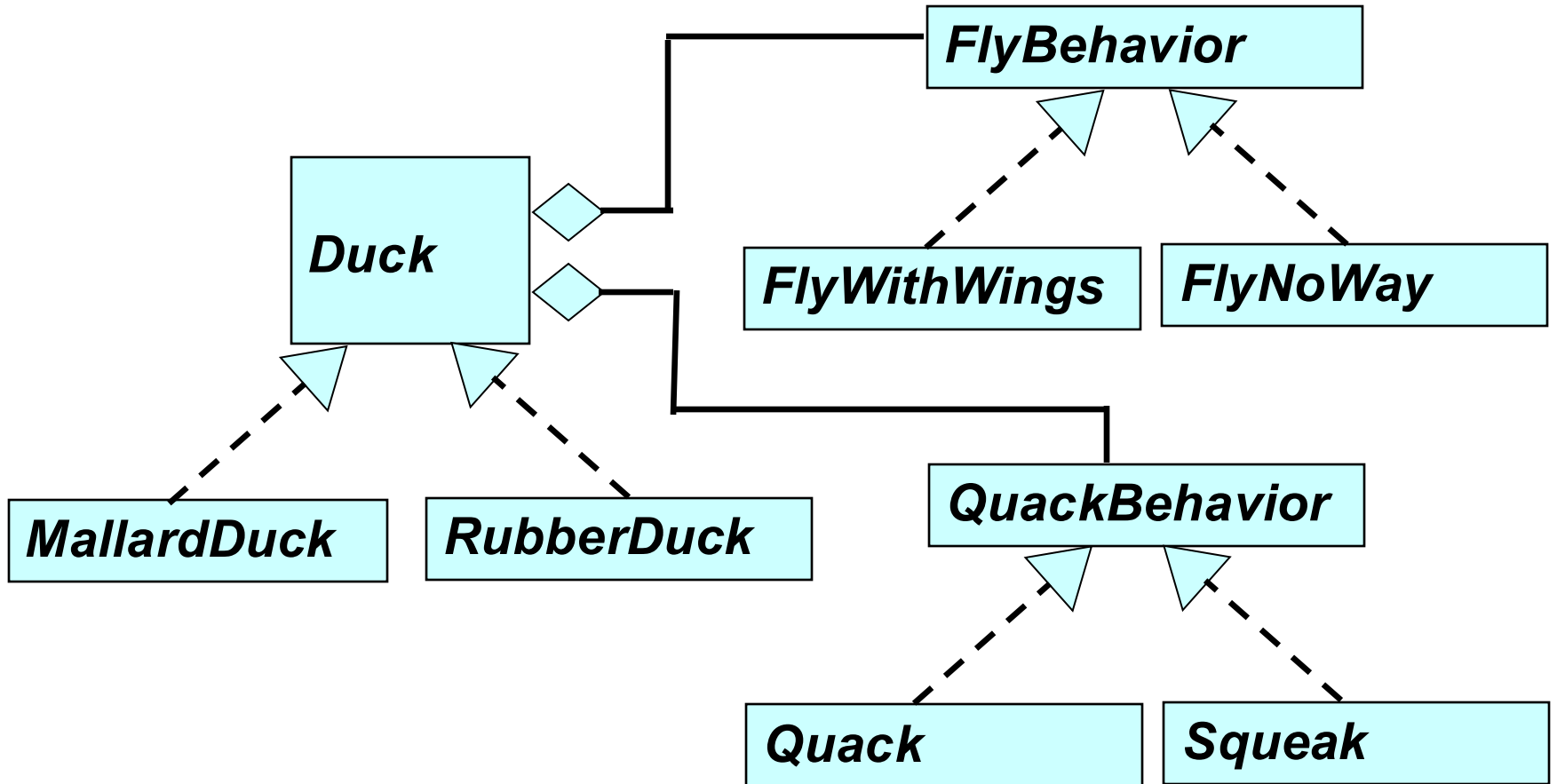
- Now we have three inheritance hierarchies for three aspects of change
 - for a) display, b) flying, and c) quacking
 - how do we link them together to specify which duck flies and which does not, which duck quacks and which one squeaks?
- The answer is: through aggregation
 - each Duck object has its own FlyBehavior object: either FlyWithWings or FlyNoWay
 - also it has its own QuackBehavior object: either of type Quack or of type Squeak

Link Hierarchies with Aggregation

- Here is an object diagram for four concrete objects



Link Hierarchies with Aggregation



Link Hierarchies with Aggregation

- We separated the Duck classes from variations in `fly()` and `quack()` behaviors
 - one advantage: these behavior objects can be reused as components of other classes
 - another advantage: new behaviors could be added to the Duck hierarchy of classes without modifying existing behavior classes
- Principle of programming to an interface
 - interface references can point to any concrete object: Duck subclasses do not know what objects they use, concrete objects implement superclass interfaces

Code to Interface vs Implementation

- Programming to 'interfaces' here means specifying the required set of methods and their signatures: e.g., fly(), quack()
 - it can be done either with a Java interface or with an abstract class: same results
 - we use type substitution: we define fields as 'supertype' references, but they point to 'subtype' objects (concrete implementation)
 - the class that defines the references does not know what type of object will be used
 - here, it is known to subclass constructor

How Easy Is It to Extend?

- If we need to add another aspect that varies among the Duck subclasses
 - We do not expand the Duck hierarchy
- Instead, we add another hierarchy of behavior classes, e.g., a PeckBehavior interface and classes that implement it
 - Then we add another component (interface reference) to the Duck superclass
- Total Duck object behavior is composed from its concrete component objects

Two Diagnostics For Good Design

- Single Responsibility Principle (**SRP**)
 - Each class should be responsible for one thing (capability, entity, computation, etc.)
 - Can phrase this as “mind your own business”
 - Object do its own calculations
 - Object should not do calculations for another
 - Easy to violate this because objects need to be connected to one another
 - e.g., Events happen as part of Dates
- Don't Repeat Yourself (**DRY**)
 - Each computational idea should be expressed just once
 - Violations often the result of
 - Cut-and-paste programming
 - Incomplete class (others have to do calculations for it, which also violates SRP)
 - But also over-specialization of classes (class = object)
- Separate Variants from Invariants

Where we stopped

Design Patterns

- High level ideas on how to structure classes and objects to solve certain problem. We need write our own code to adopt those patterns.
- Provide a shared vocabulary
- Framework and libraries are not design patterns but are specific implementations that can be used in our code
 - However, they may make use of design patterns in their implementations

The Motivation for Patterns

- We use the concepts of encapsulation, abstraction, inheritance, polymorphism
 - they alone do not lead to design that is flexible, reusable, scaleable, maintainable
 - we should identify what might change and encapsulate it from the rest of the system
 - limit the use inheritance, use composition
 - code to interfaces, not to implementation
- These are the concepts that are often used in object-oriented design

The Motivation for Patterns

- To facilitate the use of these techniques software industry developed a concept of design patterns
 - they are based on best practices of experienced designers in many application areas
 - they are not coding idioms or programming guidelines or reusable coding patterns
- We define relationships between classes regardless of their internal detail

The Origin of Design Patterns

- We use Design Pattern names rather than describe class relationships
 - this helps designers to communicate their ideas effectively
- The GoF book was the first to catalog design patterns observed most often
 - they claim that they borrowed their approach to design from architecture
 - in architecture, patterns describe typical problems and solutions to these problems that are repeated in many projects

The Origin of Design Patterns

"Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice."

*Christopher Alexander, "The Timeless Way of Building"
as quoted by Gamma et al.*

The Origin of Design Patterns

- Experienced software designers use general principles in building programs
 - but they also try to reuse solutions that proved to be successful
 - "successful" means that the design is flexible, understandable, modifiable, expandable, scalable, and maintainable
 - that is, consists of relatively independent cohesive pieces encapsulating specific levels of abstraction
 - some add to this the concept of "elegance"

The GoF Pattern Catalogue

- In their catalog, the GoF classified patterns in three broad categories
 - creational, structural, behavioral
 - the Strategy Pattern is a behavioral pattern
- Another way to classify patterns is whether they describe relationships among classes or among objects
- Creational patterns describe the process of object creation
 - they defer some work of object creation to subclasses or to other objects

The GoF Pattern Catalogue

- Structural patterns describe structural (compile time rather than runtime) relationships of classes or objects
 - as class patterns, they use inheritance to define classes; as object patterns, they describe ways to assemble objects
- Behavioral patterns describe ways in which classes or objects interact
 - for classes, they rely on inheritance to support the algorithms, for objects, they rely on general association or aggregation

The GoF Pattern Catalogue

- The GoF used a combination of four elements to describe design patterns
 - for each pattern, they specified the name, the problem, its solution and consequences
- The pattern name: a cryptic analogy
- The problem: when to apply the pattern
- The solution: the elements of design
 - their arrangement in most abstract but broadly applicable form
- The consequences: results, tradeoffs

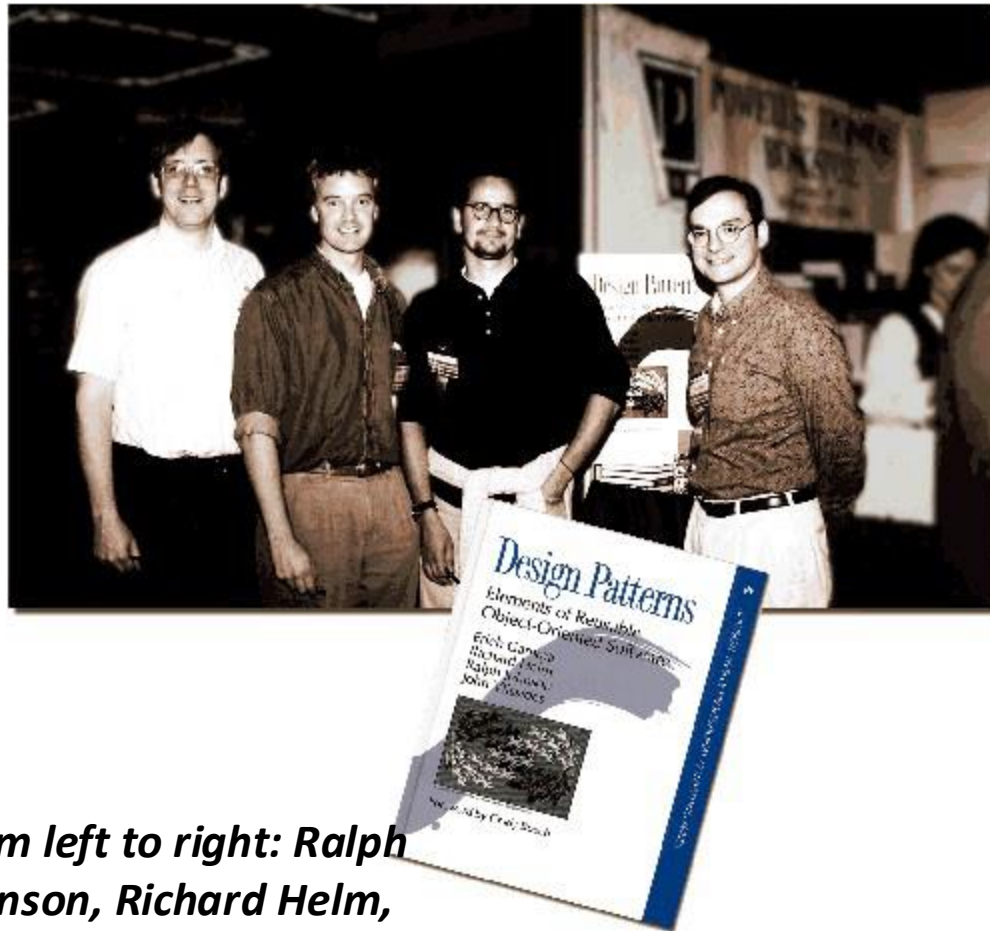
The GoF Pattern Catalogue

- As the catalogue framework, they used a super-pattern to describe patterns
 - pattern name and class to which it belongs
 - intent: what design issues does it address?
 - also known as: names used by others
 - motivation: an example that illustrates the problem and how the pattern solves it
 - applicability: situations where it is helpful
 - structure: class and interaction diagrams
 - participants: classes and objects used
 - collaborations: how participants do the work

The GoF Pattern Catalogue

- Five more bullets to go
 - consequences: how the pattern achieves its goal, the trade-offs, costs and benefits, what aspects can be varied independently
 - implementation: pitfalls, language issues
 - sample code: fragments to illustrate ideas
 - known uses: domains where the pattern was used successfully
 - related patterns: other patterns related to this one, differences and similarities

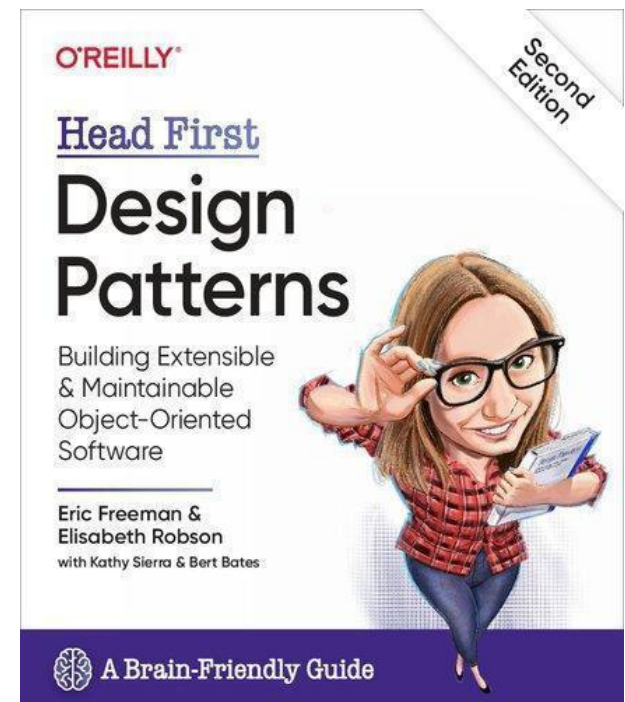
References



From left to right: Ralph Johnson, Richard Helm, Erich Gamma, and John Vlissides. (Gang of Four)

http://en.wikipedia.org/wiki/Design_Patterns

Fun review
and some
good
examples



Eric Freeman,
Elisabeth Robson

An Example Design Pattern

MVC Architecture

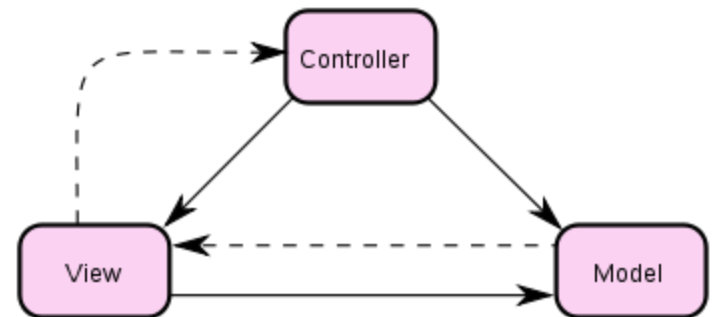
- Separate the business logic and application data from the presentation data to the user.
- Model: business logic, application data
- View: presentation of model, UI
- Controller: mediate between model and view
 - Intercept the request from view and pass to the model for proper actions.
 - Notify the views of the changes in the model.
- Model and view are independent to each other.
- In GUIs, the views and the controllers often work very closely together.
- Reusable and expressive.

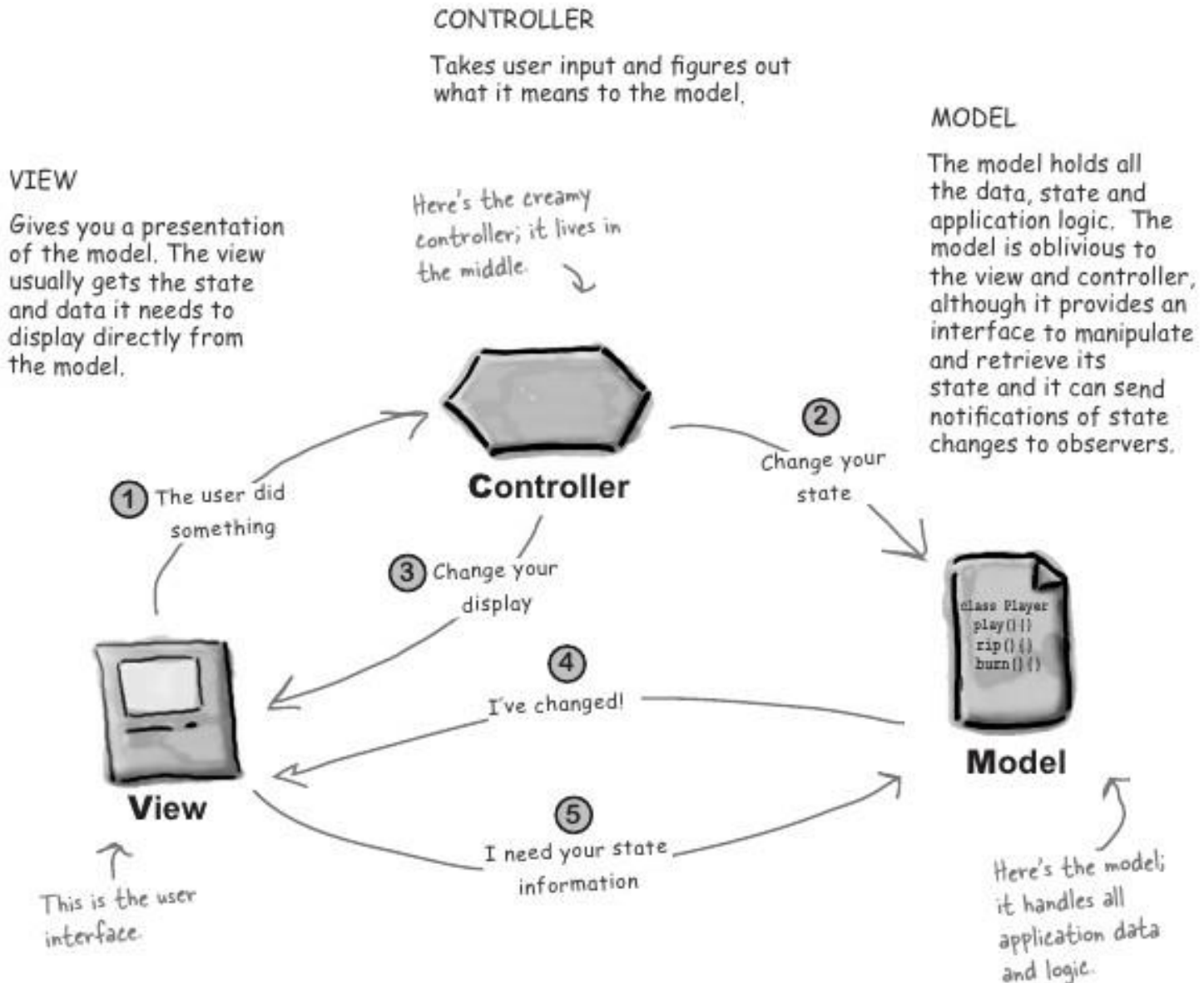
MVC Architecture

- Separate the business logic and application data from the presentation data to the user.
- Model: business logic, application data (entity objects)
- View: presentation of model, UI (boundary objects)
- Controller: (control objects)
 - Intercept the request from view (UI) and pass to the model for proper actions.
 - Notify the views of the changes in the model.
- Model and view are independent to each other.
- In GUIs, the views and the controllers often work very closely together.
- Reusable and expressive.

MVC

- First implemented by Trygve Reenskaug at 1979 on Smalltalk at Xerox labs.
- Used widely
 - GUI, web applications
- Compound Design pattern
- Used in a lot of frameworks
 - Java swing, ruby on rails

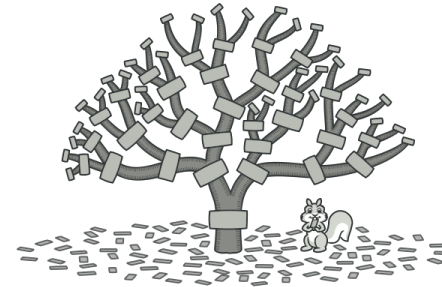
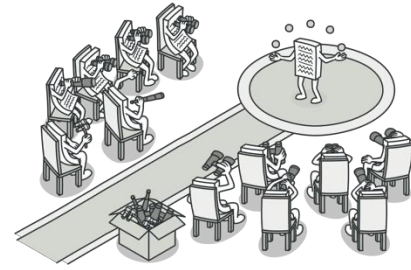




From "Head First Design Patterns"

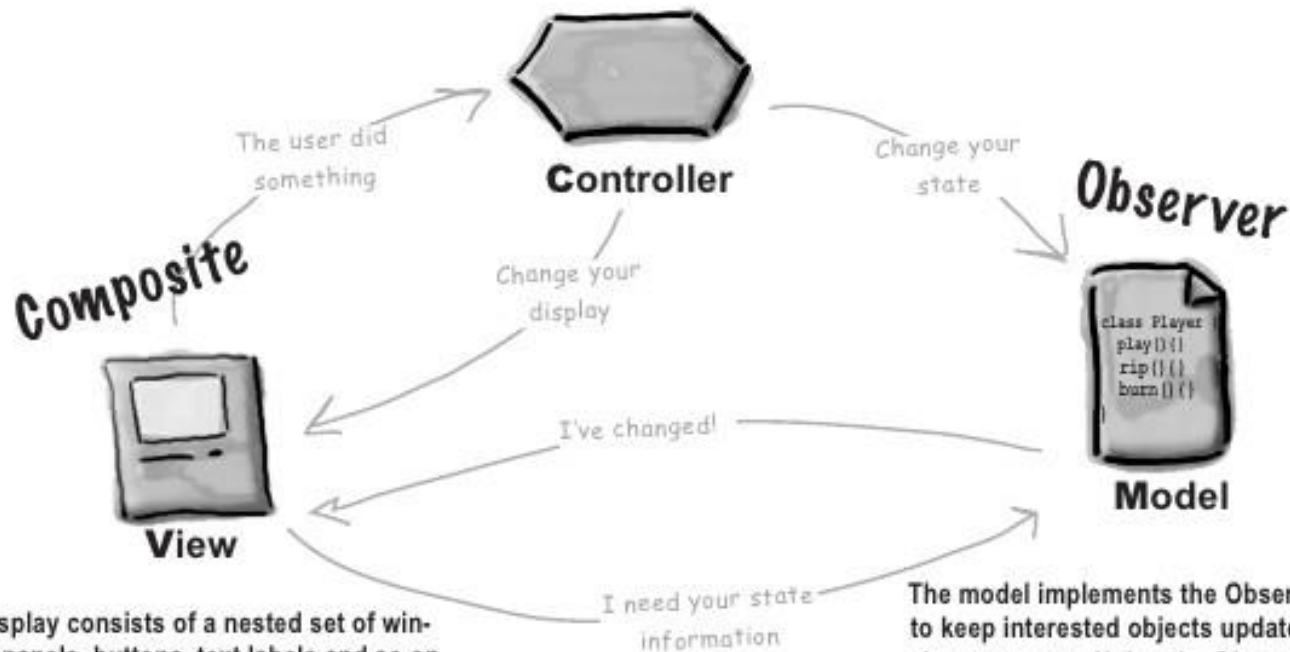
MVC Patterns

- Compound design pattern
 - Observer Pattern
 - Strategy Pattern
 - Composite Pattern



Strategy

The view and controller implement the classic Strategy Pattern: the view is an object that is configured with a strategy. The controller provides the strategy. The view is concerned only with the visual aspects of the application, and delegates to the controller for any decisions about the interface behavior. Using the Strategy Pattern also keeps the view decoupled from the model because it is the controller that is responsible for interacting with the model to carry out user requests. The view knows nothing about how this gets done.

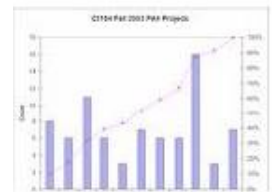
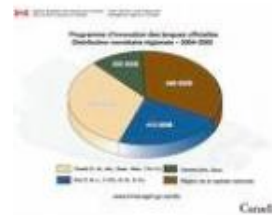
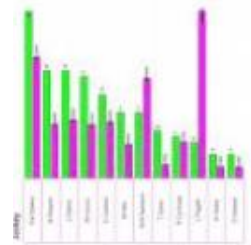
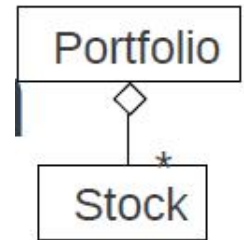


The display consists of a nested set of windows, panels, buttons, text labels and so on. Each display component is a composite (like a window) or a leaf (like a button). When the controller tells the view to update, it only has to tell the top view component, and Composite takes care of the rest.

The model implements the Observer Pattern to keep interested objects updated when state changes occur. Using the Observer Pattern keeps the model completely independent of the views and controllers. It allows us to use different views with the same model, or even use multiple views at once.

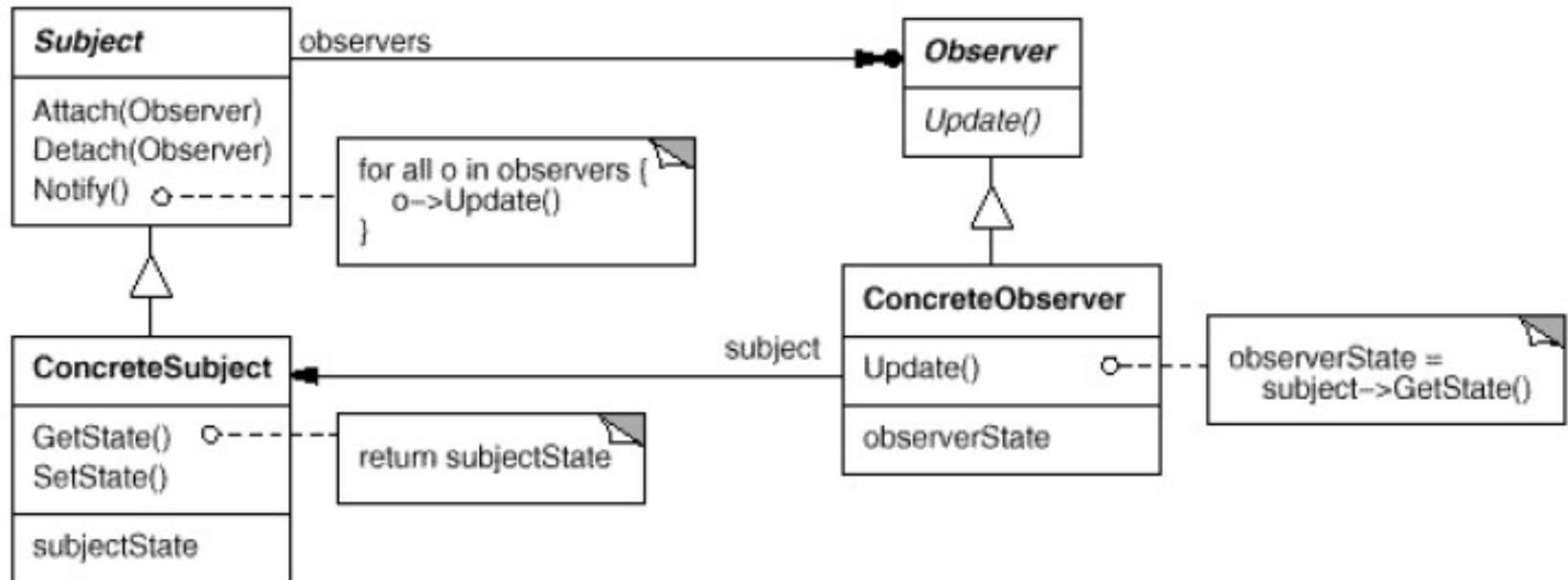
Observer Pattern Motivation

- We have an object that changes its state quite often
 - Example: A Portfolio of stocks
- We want to provide multiple views of the current state of the portfolio
 - Example: Histogram view, pie chart view, time line view, alarm
- Requirements:
 - The system should maintain consistency across the (redundant) views, whenever the state of the observed object changes
 - The system design should be highly extensible
 - It should be possible to add new views without having to recompile the observed object or the existing views.



Observer Pattern

- When you need to write a dependency between objects so that one object can notify its (many) dependents of a state change
- Weather station; Stock report update



3 Possibilities to change the file name

Example

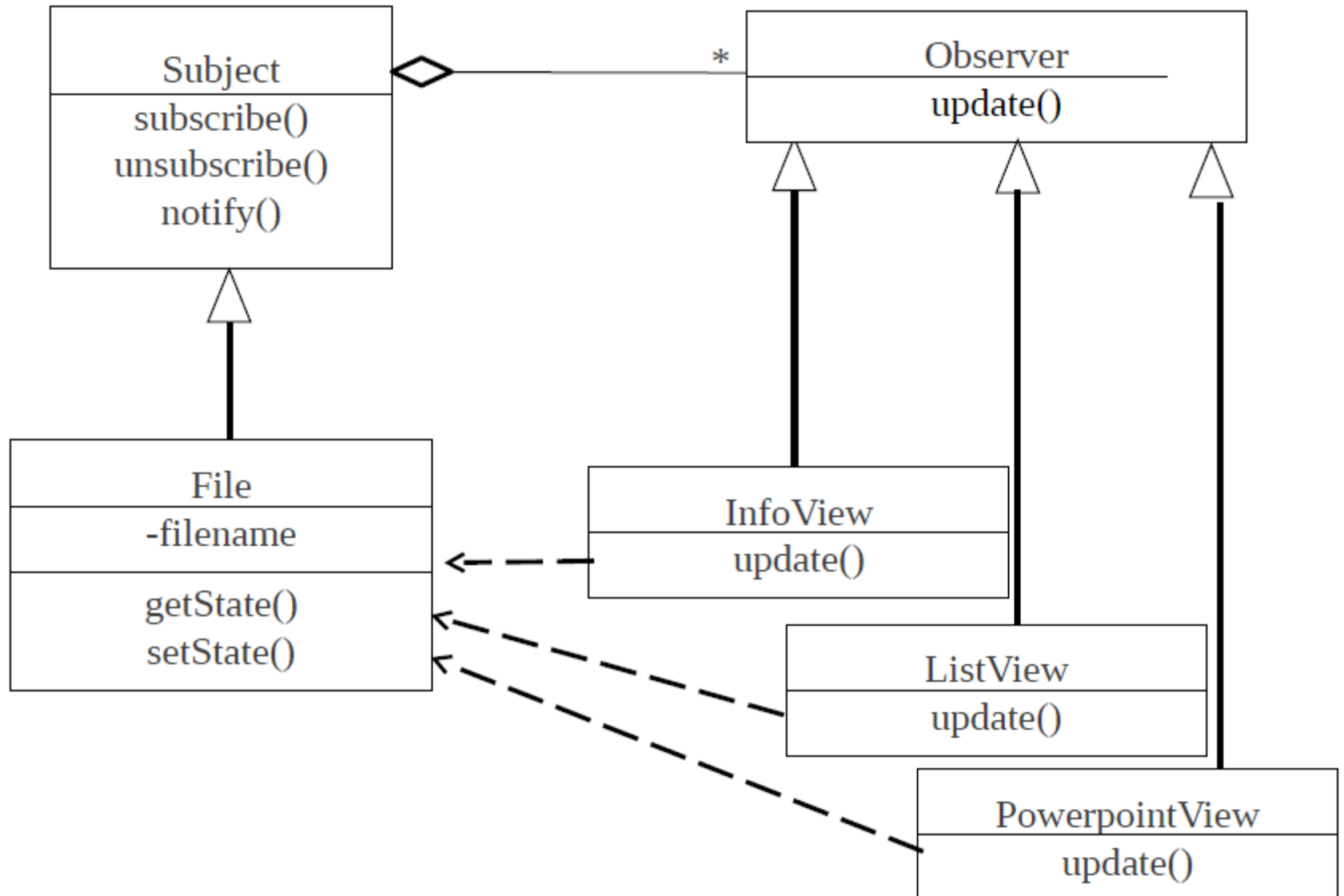
The screenshot shows a Mac OS X desktop environment. On the left is the 'L9_DesignPatterns2.ppt Info' window. The 'General' tab is selected, showing file details: 'L9_DesignPatterns2.... 3,7 MB', 'Modified: Today at 10:03', 'Kind: Microsoft PowerPoint document', 'Size: 3,7 MB on disk (3.833.856 bytes)', 'Where: /Users/berndbruegge/Teaching/ SS 2007 Software Engineering I (EIST)/Lectures/ 5 Design Patterns', 'Created: Mittwoch, 16. Mai 2007 10:49', 'Modified: Today at 10:03', and 'Color label:'. The 'Name & Extension' field is highlighted with a red box and contains 'L9_DesignPatterns2.ppt'. A blue callout bubble labeled 'Info View' points to this field.

In the center is the 'Lectures' folder window in 'List View'. It shows a list of files: '5 Design Patterns', 'L9_DesignPatterns2.ppt', and 'Bumpers'. The file 'L9_DesignPatterns2.ppt' is highlighted with a red box. A blue callout bubble labeled 'List View' points to this window.

On the right is the PowerPoint window showing a slide titled 'Example: The File Name of a Presentation' and '3 Possibilities to change the File Name'. The slide content includes a blue box with the text 'What happens if I change the file name of this presentation in List View to foo?'. A blue callout bubble labeled 'PowerPoint View' points to the slide.

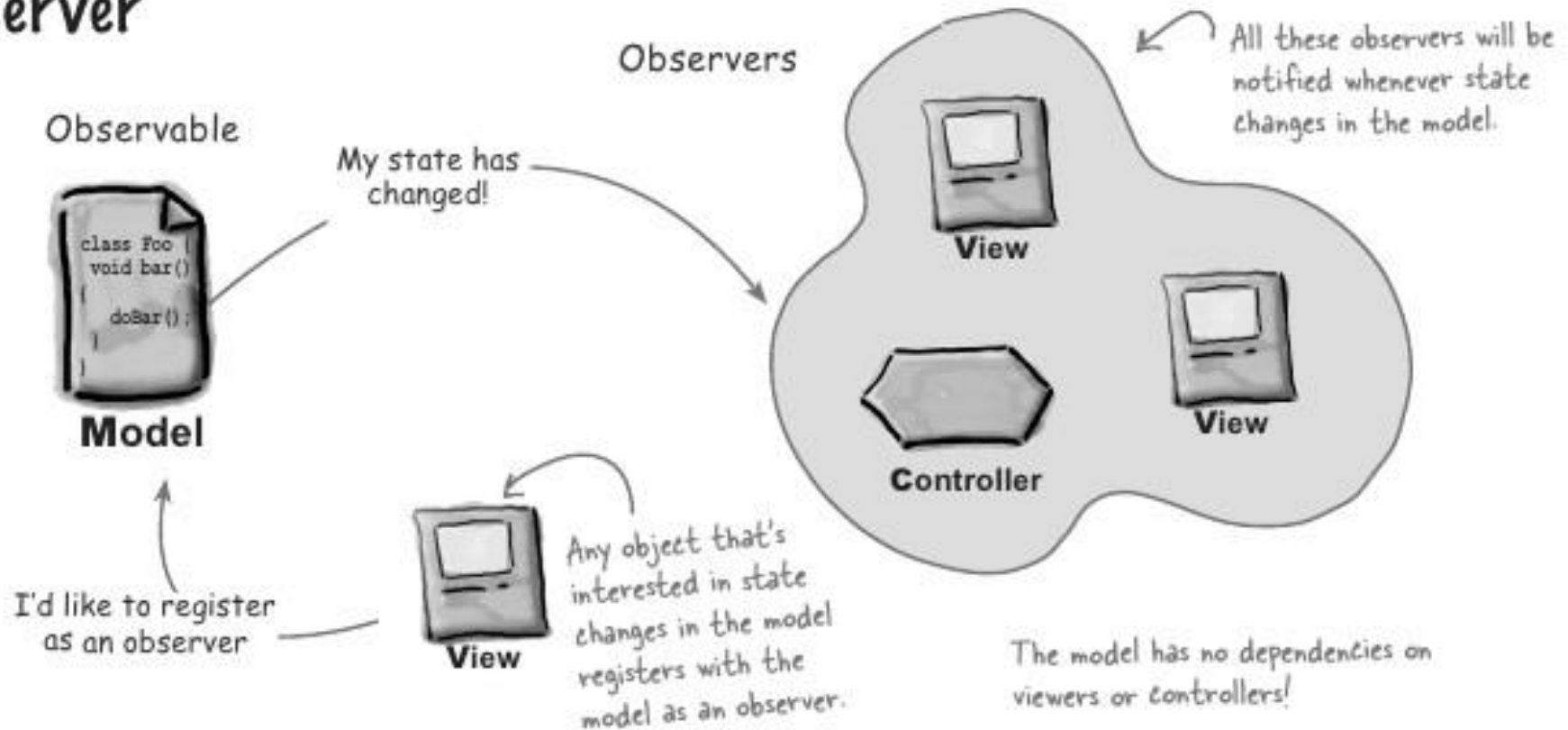
The slide footer contains the text '© 2007 Bernd Bruegge', 'Introduction into Software Engineering Summer 2007', and 'Slide 59 of 90'.

Example



Observer

Observer

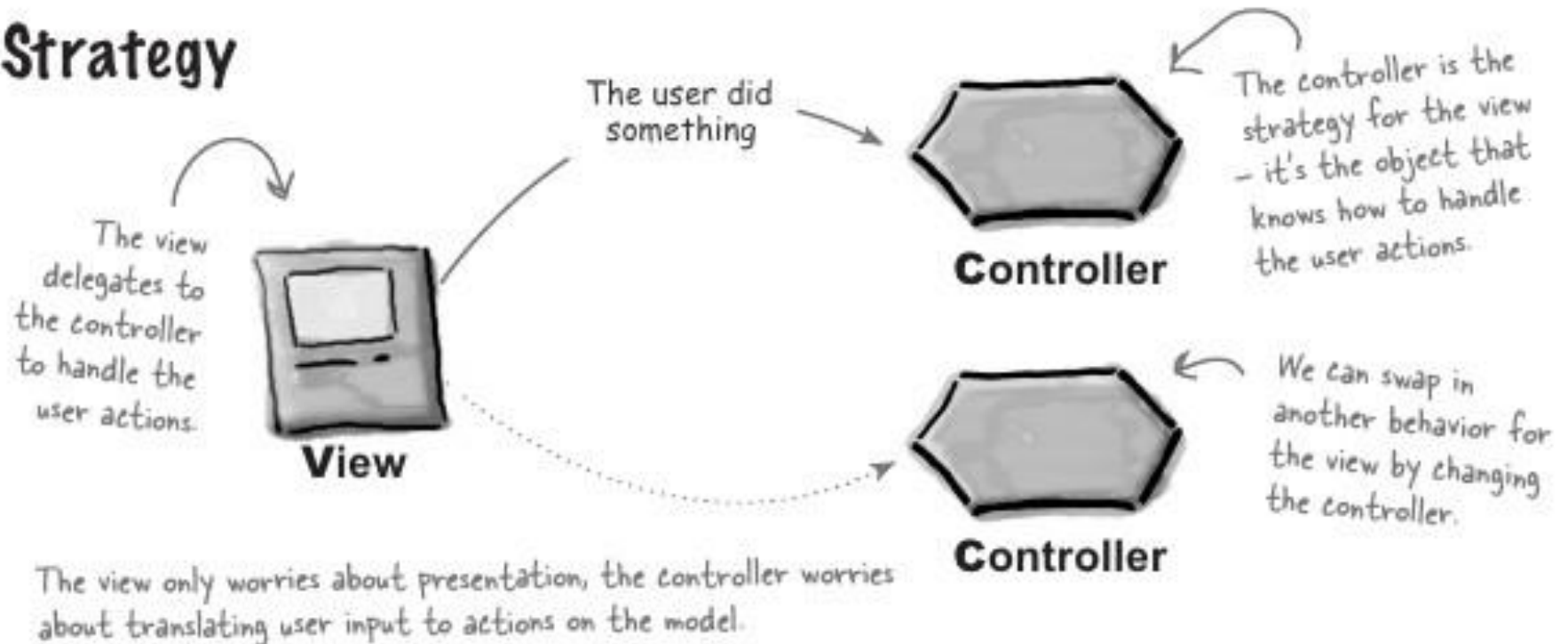


Strategy Pattern Motivation

- Different algorithms exist for a specific task, we can switch between the algorithms at run time.
- Examples of tasks:
 - Different collision strategies for objects in video games
 - Parsing a set of tokens into an abstract syntax tree (Bottom up, top down)
 - Sorting a list of customers (Bubble sort, mergesort, quicksort)
- Different algorithms will be appropriate at different times
- If we need a new algorithm, we can add it without disturbing the application or the other algorithms.
- Strategy provides a way to configure a class with one of many behaviors.

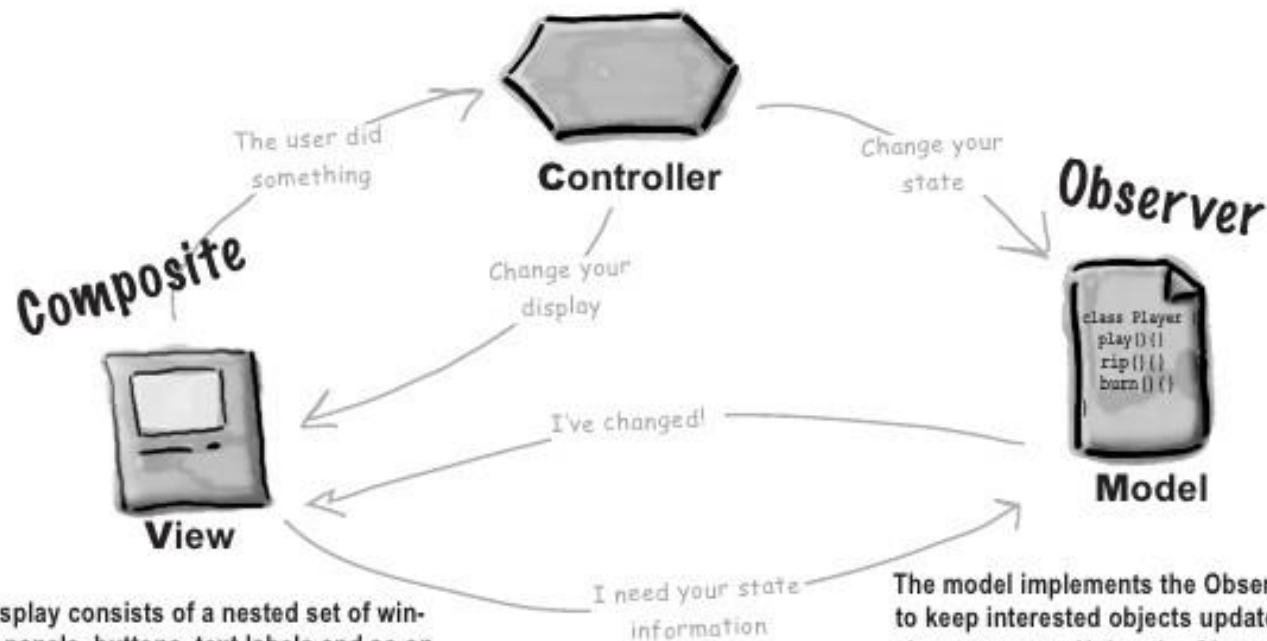
Strategy

Strategy



Strategy

The view and controller implement the classic Strategy Pattern: the view is an object that is configured with a strategy. The controller provides the strategy. The view is concerned only with the visual aspects of the application, and delegates to the controller for any decisions about the interface behavior. Using the Strategy Pattern also keeps the view decoupled from the model because it is the controller that is responsible for interacting with the model to carry out user requests. The view knows nothing about how this gets done.



The display consists of a nested set of windows, panels, buttons, text labels and so on. Each display component is a composite (like a window) or a leaf (like a button). When the controller tells the view to update, it only has to tell the top view component, and Composite takes care of the rest.

The model implements the Observer Pattern to keep interested objects updated when state changes occur. Using the Observer Pattern keeps the model completely independent of the views and controllers. It allows us to use different views with the same model, or even use multiple views at once.

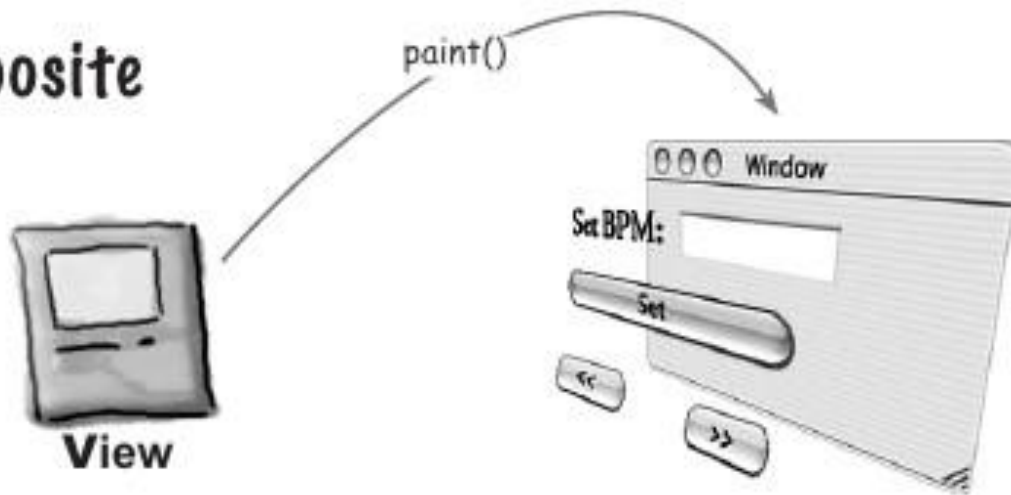
From "Head First Design Patterns"

Composite Pattern Motivation

- Software System:
 - Definition: A software system consists of subsystems which are either other subsystems or collection of classes
 - Composite: Subsystem (A software system consists of subsystems which consists of subsystems , which consists of subsystems, which...)
 - Leaf node: Class
- Models tree structures that represent part-whole hierarchies with arbitrary depth and width.
- The Composite Pattern lets client treat individual objects and compositions of these objects uniformly

Composite

Composite



The view is a composite of GUI components (labels, buttons, text entry, etc.). The top level component contains other components, which contain other components and so on until you get to the leaf nodes.

MVC in Web Applications

- Model: application domain data or business logic. (Often directly related to DB, knowing nothing about html, or web server)
- View: UI. Usually can query model to display, but not supposed to change model. It can be implemented using a template that renders an html page
- Control: handle user actions. Receives user requests and translate them into actions that model should take. It can also select appropriate view to handle the response – it can handle the incoming http requests. (e.g. common actions like authentication, session management)
- Decouple model and view, so that the presentation is always independent of application data
- Decouple view and control so that the request made by the user can be independent of the resulting page.

MVC Web Framework

- ASP .NET Framework (...)
- Spring framework (...) (Java)
- CodeIgniter (...) (php)
- Ruby on Rails (...) (Ruby)
- Django (...) (Python)
- http://en.wikipedia.org/wiki/Comparison_of_web_application_frameworks

MVC Web Applications in PHP

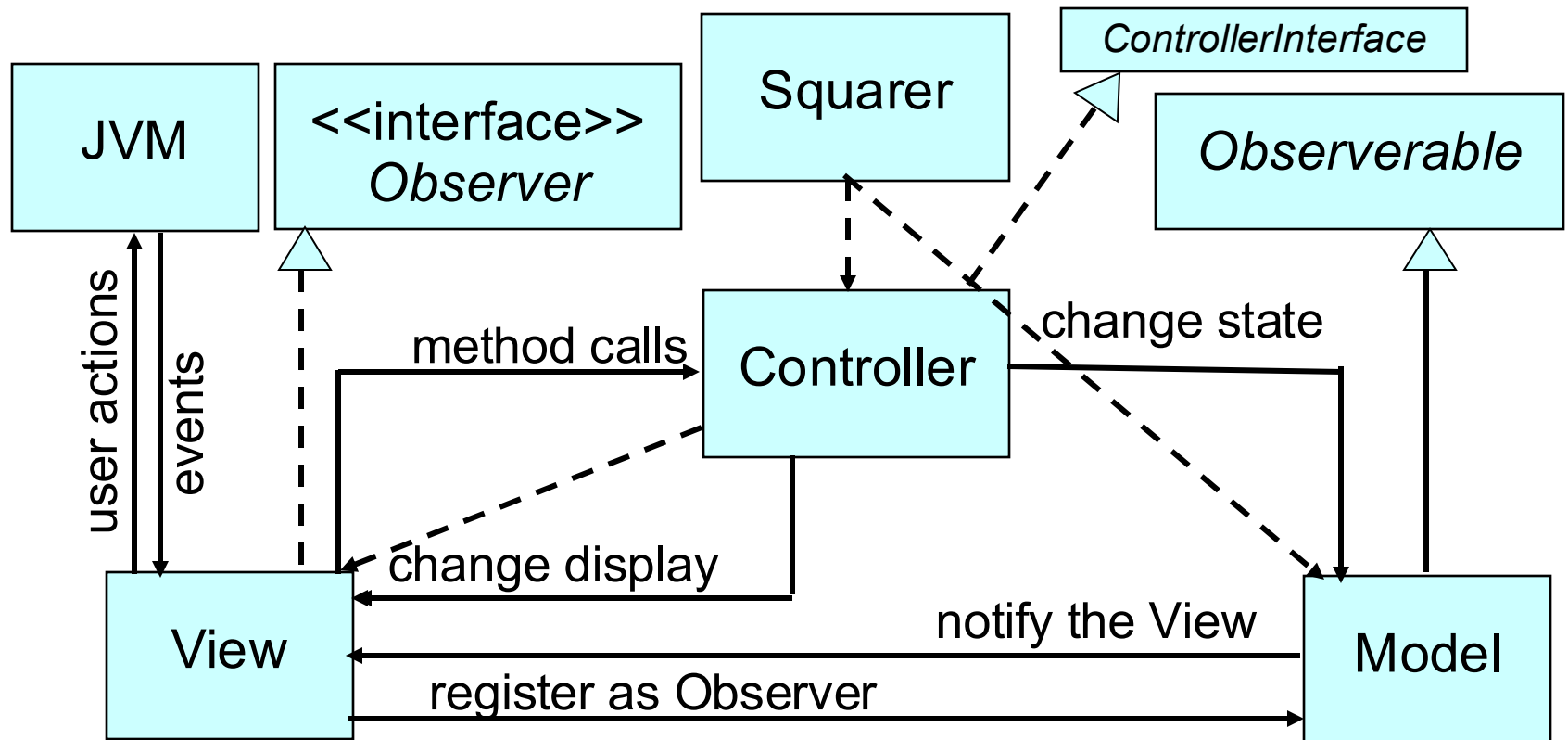
- Model – those functions that wrap calls to your db
- View – the templates / scripts that output HTML
- Controller – the stuff that examines variables like `$_GET` and `$_POST` and works out what to do next

Controller

- Controller should be as minimal as possible.
 - It is only responsible for translating HTTP requests into Model actions and selecting the right View
 - The Model should provide all the behavior it can without handling the HTTP requests or output formatting details

The MVC Pattern: Summary

- The UML diagrams for this design:
 - user actions are passed from the View to JVM, back to View, then to the Controller

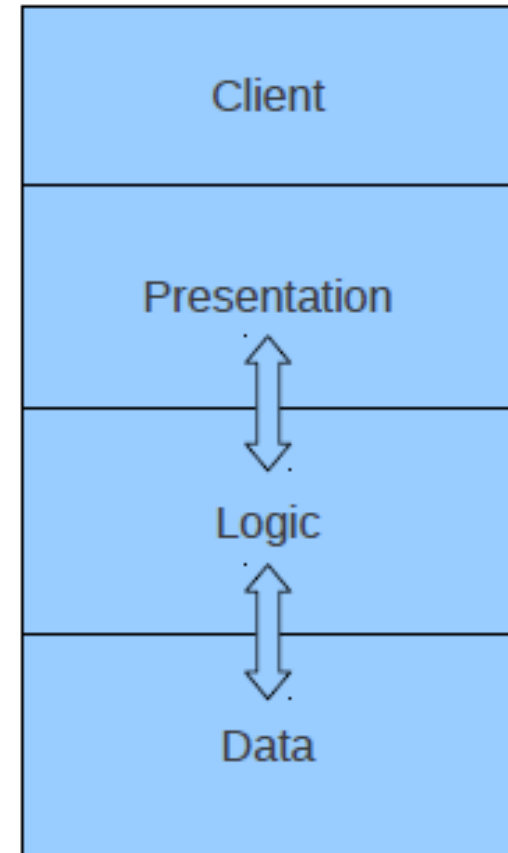


The MVC Pattern: Summary

- Summary
 - the MVC compound pattern has the Strategy, Observer, and Composite patterns with the Adapter as a possibility
 - the Controller is the strategy for the view (different implementations can be used)
 - the Model represents the problem domain; it notifies its observer View but is decoupled from it and from the Controller
 - the View is a composite of nested composites and their components

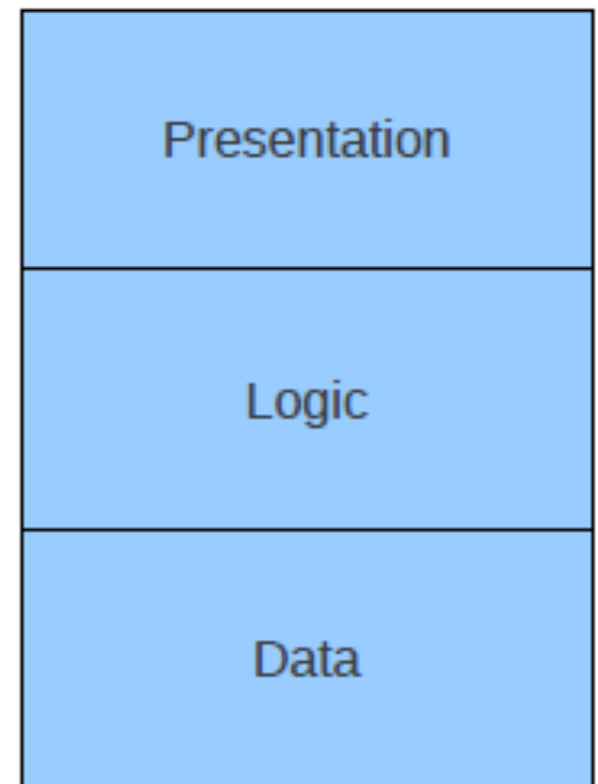
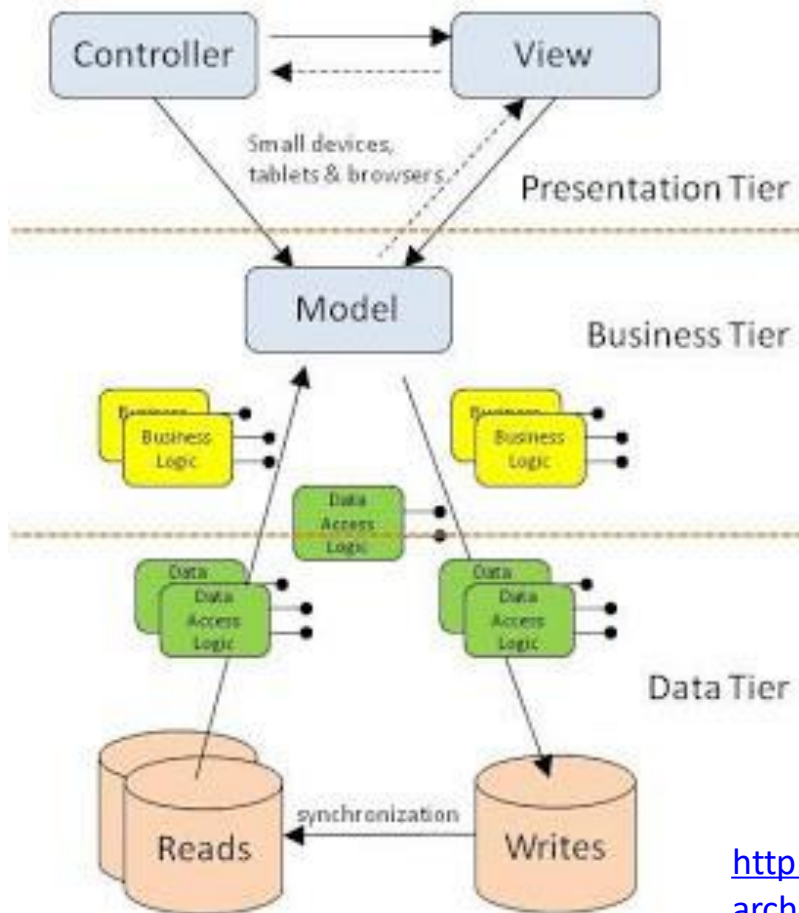
Three/Four -Tier Architecture

- Presentation: UI
- Logic: business logic
- Data: database/file
- Performance, security, reuse, maintainability and understandability.



MVC vs. Three-Tier Architecture

- Three-tier Architecture



<http://criticaltechnology.blogspot.com/2011/10/mvc-in-three-tier-architecture.html>